Data Mining 2026 — Project Notebook

Course: Data Mining
Project track: [x] Standard Analysis [] Research-Oriented
Group members:

- Rasmus Skovgaard Pedersen
- Hjalte Vilford Vinther
- Nour Mamoun Abdul Rahman Al Said

Dataset: Online Retail
GitHub: GitHub Link

Project Task

The objective of this project is to analyze transactional retail data in order to identify meaningful purchasing patterns, customer behavior, and product relationships. The analysis focuses on uncovering both transaction-level structure and product-level associations within the dataset.

First, invoice-level features are aggregated and used for clustering to identify distinct transaction segments. This allows similar purchasing behaviors to be grouped together and helps determine which types of transactions contribute most to overall revenue.

Second, outlier analysis is applied to detect extreme revenue transactions and examine whether a small number of transactions account for a disproportionate share of revenue.

Graph mining is then used to model relationships between products as a network, where products represent nodes and co-occurrence relationships represent edges. This allows structural patterns and strongly connected product groups to be identified, and provides an additional perspective for comparing transaction structure with anomalous purchasing behavior.

Finally, frequent itemset mining and association rule analysis are applied to identify products that are frequently purchased together and to uncover localized purchasing dependencies. These techniques complement graph-based analysis by revealing statistical relationships between products that may support recommendation systems, bundled promotions, and cross-selling strategies.

0. Reproducibility and Setup

This notebook demonstrates clustering, analysis, graph mining and pattern mining of the Online Retail dataset.

All steps are reproducible with the provided random seed (*42*).

In [ ]:
```
# Packages that might need to be installed
%pip install networkx
%pip install community
%pip install python-louvain
%pip install kagglehub
%pip install mlxtend
```

In [96]:
```
import sys
print(sys.version)

# Hide warnings when running code
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import seaborn as sns
import statsmodels.api as sm
import scipy.cluster.hierarchy as sch
import networkx as nx
import community.community_louvain as community_louvain
import kagglehub
from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import RobustScaler
from scipy.stats import zscore
from itertools import combinations
from collections import defaultdict
from networkx.algorithms.community import k_clique_communities
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

```
3.11.15 | packaged by conda-forge | (main, Mar  5 2026, 16:36:00) [MSC v.1944 64 bit (AMD64)]
```

1. Dataset Description and Loading

1.1 Dataset Overview

Dataset: Online Retail

Description: The dataset contains all transactions occurring between $01/12/2010$ and $09/12/2011$ for a UK-based online retail store.

Features:

- `InvoiceNo` : Invoice number, a 6-digit integral number uniquely assigned to each transaction.
- `StockCode` : Product (item) code.
- `Description` : Product name.
- `Quantity` : The quantities of each product per transaction.
- `InvoiceDate` : Date and time of invoice.
- `UnitPrice` : Price per unit (in GBP).
- `CustomerID` : Unique customer identifier.
- `Country` : Country of the customer.

Additional Data

- Number of instances: 541909
- Missing values: 136534 (spread out between `CustomerID` and `Description`)

Below is a quick look at the dataset:

In [3]:
```
# Load dataset directly from kaggle
path = kagglehub.dataset_download("tunguz/online-retail")
csv_path = os.path.join(path, 'Online_Retail.csv')

data = pd.read_csv(csv_path,
            encoding='ISO-8859-1')

data.head()
```

Out[3]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	C
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/10 8:26	2.55	17850.0	K
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	17850.0	K
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	17850.0	K
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	17850.0	K
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	17850.0	K

2. Cleaning and Transformation

During the initial examination of the dataset, it was observed that several *invoices* contained negative quantities. In some cases, the `InvoiceNo` begins with a C, indicating that the transaction was cancelled. These represent legitimate cancellations and are therefore kept. However, most negative quantities correspond to products that were damaged or destroyed and therefore discarded. Such transactions are not representative of normal customer activity and are therefore removed from the analysis.

Furthermore, a couple of invoices begins with an A, and has the description `bad dept`. These entries do not correspond to valid transactions and are also excluded from the dataset.

Following these cleaning steps, the dataset is prepared for subsequent analyses, including basic statistical summaries, distributional assessments, and an evaluation of sparsity and density.

So in this step, we remove invoices that are not representative of typical customer transactions.
Specifically:

1. Invoices with negative quantities, except for those starting with C (cancelled orders).
2. Invoices starting with A and containing placeholder descriptions (e.g., `bad dept`).

In [5]:
```
# Identifies all the invoices with negative quantities:
negative_invoices = data[data["Quantity"] < 0]["InvoiceNo"].unique()

# Exclude cancelled orders (InvoiceNo starting with 'C') from removal
not_cancelled_negative_invoices = [invoice for invoice in negative_invoices if n

# Identify invoices starting with 'A'
starts_with_a = data[data["InvoiceNo"].str.startswith("A")]["InvoiceNo"].unique(

# Remove invalid invoices from the dataset
data_filtered = data[~data["InvoiceNo"].isin(not_cancelled_negative_invoices)]
data_filtered = data_filtered[~data_filtered["InvoiceNo"].isin(starts_with_a)]

# Save the cleaned dataset for downstream analysis as a csv file
cleaned_data_path = os.path.join("data", "processed", "Online_Retail_Cleaned.csv
data_filtered.to_csv(cleaned_data_path, index=False)

# Sanity checks
amount_to_remove = len(not_cancelled_negative_invoices) + len(starts_with_a)
print("Amount of rows to remove:", amount_to_remove)
assert data.shape[0] - amount_to_remove == data_filtered.shape[0], "The number o
```

```
Amount of rows to remove: 1339
```

In [6]:
```
data_clean = data_filtered[
    (data_filtered["Quantity"] > 0) &
    (~data_filtered["InvoiceNo"].astype(str).str.startswith("C"))
].copy()

# Sanity check
print("Remaining cancelled rows:", data_clean["InvoiceNo"].astype(str).str.start
assert data_clean["Quantity"].min() > 0

data_complete_orders_path = os.path.join(
    "data", "processed", "Online_Retail_Complete_Orders.csv"
)

data_clean.to_csv(data_complete_orders_path, index=False)

print("Final dataset shape:", data_clean.shape)
```

```
Remaining cancelled rows: 0
Final dataset shape: (531282, 8)
```

3. Basic Statistics and Distribution Analysis

After cleaning the dataset, we now examine its structure and distributions to better understand the data. This includes:

1. Shape and missing values.
2. Summary statistics for numerical features (`Quantity` , `UnitPrice`).
3. Distribution of key categorical features (`Country` , `CustomerID`).
4. Visualizations for `Quantity` and `UnitPrice` to detect skewness and outliers.
5. Sparsity and Density

3.1 Shape and Missing Values

To begin, we examine the structure of the dataset by comparing the number of entries and missing values in the raw dataset with those in the cleaned dataset. This allows us to verify the effectiveness of our cleaning steps and ensures that we have a complete and consistent dataset for subsequent analyses.

In [7]:
```
#First lets look at the number of rows and columns, as well as the datatypes, as
print("Original data:")
print("Shape:", data.shape)
#We can also look at the number of missing values in each column.
print("Missing values in original data:")
print(data.isnull().sum())

#Lets do the same for the cleaned data.
print("\nCleaned data:")
print("Shape:", data_clean.shape)
print("Missing values in cleaned data:")
print(data_clean.isnull().sum())
```

```
Original data:
Shape: (541909, 8)
Missing values in original data:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64

Cleaned data:
Shape: (531282, 8)
Missing values in cleaned data:
InvoiceNo           0
StockCode           0
Description       592
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     133358
Country             0
dtype: int64
```

Dealing with missing values

The output above shows missing values in two columns:

- **`CustomerID`** – $133,741$ values are missing after cleaning (down from $135,080$ in the raw data). Because our analysis targets the **invoice level**, every invoice row still carries complete transactional information (products, quantities, unit prices) regardless of whether a `CustomerID` is recorded. These rows are therefore

retained. `CustomerID` appears in the feature table for reference only and is not used as a clustering feature.

- `Description` – 592 values are missing after cleaning (down from $1,454$ in the raw data). `Description` is a free-text label that plays no role in feature engineering or clustering, so missing descriptions have no impact on the analysis.

All other key fields (`Quantity` , `UnitPrice` , `InvoiceDate` , `Country` , `StockCode` , `InvoiceNo`) contain **no missing values**, guaranteeing that every derived feature (`TotalPrice` , `TotalQuantity` , etc.) is computed from complete data.

Beyond the cleaning already performed — removal of non-cancelled negative-quantity invoices and 'bad debt' entries — **no further imputation or row removal is necessary**.

3.2 Statistics for Numerical Features

Next, we examine summary statistics for the numerical features, specifically `Quantity` and `UnitPrice` .
For each feature, we calculate the total, mean, standard deviation, minimum, and maximum values.

Note: The `CustomerID` column is numeric but represents identifiers rather than measurable quantities, so it is excluded from this statistical summary.

These statistics provide an initial understanding of the distribution, scale, and variability of the transaction data, and help identify potential outliers that may influence subsequent analyses.

In [8]:
```
# Statistics for original data
print("Statistics for original data:")
data_no_customer = data.drop(columns=['CustomerID'])
data_no_customer.describe()
```

```
Statistics for original data:
```

Out[8]:

	Quantity	UnitPrice
count	541909.000000	541909.000000
mean	9.552250	4.611114
std	218.081158	96.759853
min	-80995.000000	-11062.060000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	38970.000000

In [9]:
```
# Same statistics for cleaned data
print("Statistics for cleaned data:")
```

```
data_filtered_no_customer = data_clean.drop(columns=['CustomerID'])
data_filtered_no_customer.describe()
```

```
Statistics for cleaned data:
```

Out[9]:

	Quantity	UnitPrice
count	531282.000000	531282.000000
mean	10.655317	3.878140
std	156.830764	32.510663
min	1.000000	0.000000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	13541.330000

3.3 Categorical Features Overview

We now examine key categorical features to understand the diversity of the dataset.

- `Country` : Shows the distribution of transactions across countries.
- `CustomerID` : Counts the number of unique customers in the dataset.
- `StockCode` : Counts the number of unique products sold.
- `InvoiceNo` : Counts the total number of unique invoices.

This overview helps assess dataset coverage and identify dominant categories.

In [10]:
```
# Categorical overview
print(data["Country"].value_counts())
print("Unique CustomerIDs:", data["CustomerID"].nunique())
print("Unique StockCodes:", data["StockCode"].nunique())
print("Unique InvoiceNos:", data["InvoiceNo"].nunique())
```

```
Country
United Kingdom          495478
Germany                   9495
France                    8557
EIRE                      8196
Spain                     2533
Netherlands               2371
Belgium                   2069
Switzerland               2002
Portugal                  1519
Australia                 1259
Norway                    1086
Italy                      803
Channel Islands            758
Finland                    695
Cyprus                     622
Sweden                     462
Unspecified                446
Austria                    401
Denmark                    389
Japan                      358
Poland                     341
Israel                     297
USA                        291
Hong Kong                  288
Singapore                  229
Iceland                    182
Canada                     151
Greece                     146
Malta                      127
United Arab Emirates        68
European Community          61
RSA                         58
Lebanon                     45
Lithuania                   35
Brazil                      32
Czech Republic              30
Bahrain                     19
Saudi Arabia                10
Name: count, dtype: int64
Unique CustomerIDs: 4372
Unique StockCodes: 4070
Unique InvoiceNos: 25900
```

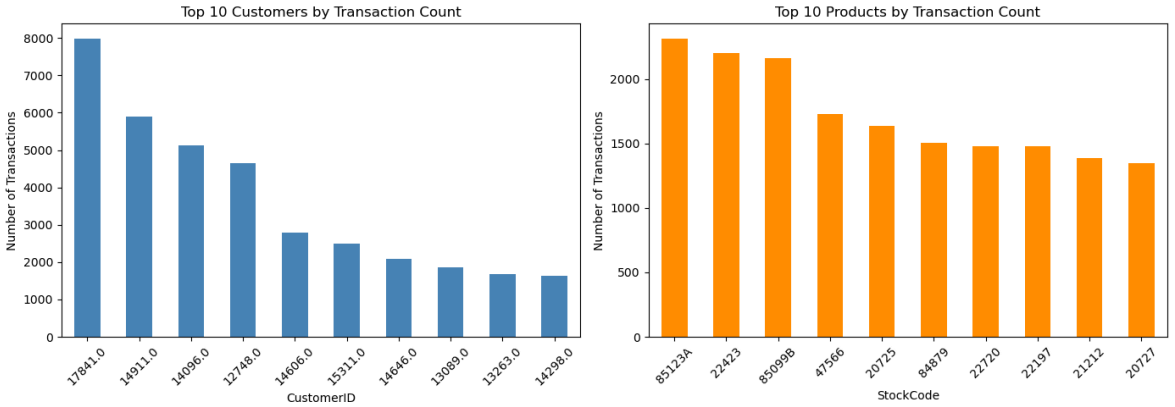
In [11]:
```
# Categorical overview after cleaning
print(data_clean["Country"].value_counts())
print("Unique CustomerIDs:", data_clean["CustomerID"].nunique())
print("Unique StockCodes:", data_clean["StockCode"].nunique())
print("Unique InvoiceNos:", data_clean["InvoiceNo"].nunique())
```

```
Country
United Kingdom          486283
Germany                   9042
France                    8408
EIRE                      7894
Spain                     2485
Netherlands               2363
Belgium                   2031
Switzerland               1967
Portugal                  1501
Australia                 1185
Norway                    1072
Italy                      758
Channel Islands            748
Finland                    685
Cyprus                     614
Sweden                     451
Unspecified                446
Austria                    398
Denmark                    380
Poland                     330
Japan                      321
Israel                     295
Hong Kong                  284
Singapore                  222
Iceland                    182
USA                        179
Canada                     151
Greece                     145
Malta                      112
United Arab Emirates        68
European Community          60
RSA                         58
Lebanon                     45
Lithuania                   35
Brazil                      32
Czech Republic              25
Bahrain                     18
Saudi Arabia                 9
Name: count, dtype: int64
Unique CustomerIDs: 4339
Unique StockCodes: 3940
Unique InvoiceNos: 20725
```

The numeric summary above tells us *how many* unique customers and products exist, but not how their transaction counts are distributed. The bar charts below show the **top 10 customers** and **top 10 products** by number of line items, revealing whether activity is concentrated in a small number of buyers or stock codes.

In [12]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Top 10 customers by transaction count (exclude NaN CustomerIDs)
top_customers = data["CustomerID"].dropna().value_counts().head(10)
top_customers.plot(kind="bar", ax=axes[0], color="steelblue")
axes[0].set_title("Top 10 Customers by Transaction Count")
axes[0].set_xlabel("CustomerID")
axes[0].set_ylabel("Number of Transactions")
axes[0].tick_params(axis='x', rotation=45)
```

```
# Top 10 products (StockCode) by transaction count
top_products = data["StockCode"].value_counts().head(10)
top_products.plot(kind="bar", ax=axes[1], color="darkorange")
axes[1].set_title("Top 10 Products by Transaction Count")
axes[1].set_xlabel("StockCode")
axes[1].set_ylabel("Number of Transactions")
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



3.4 Distributions of Quantity and Unit Price

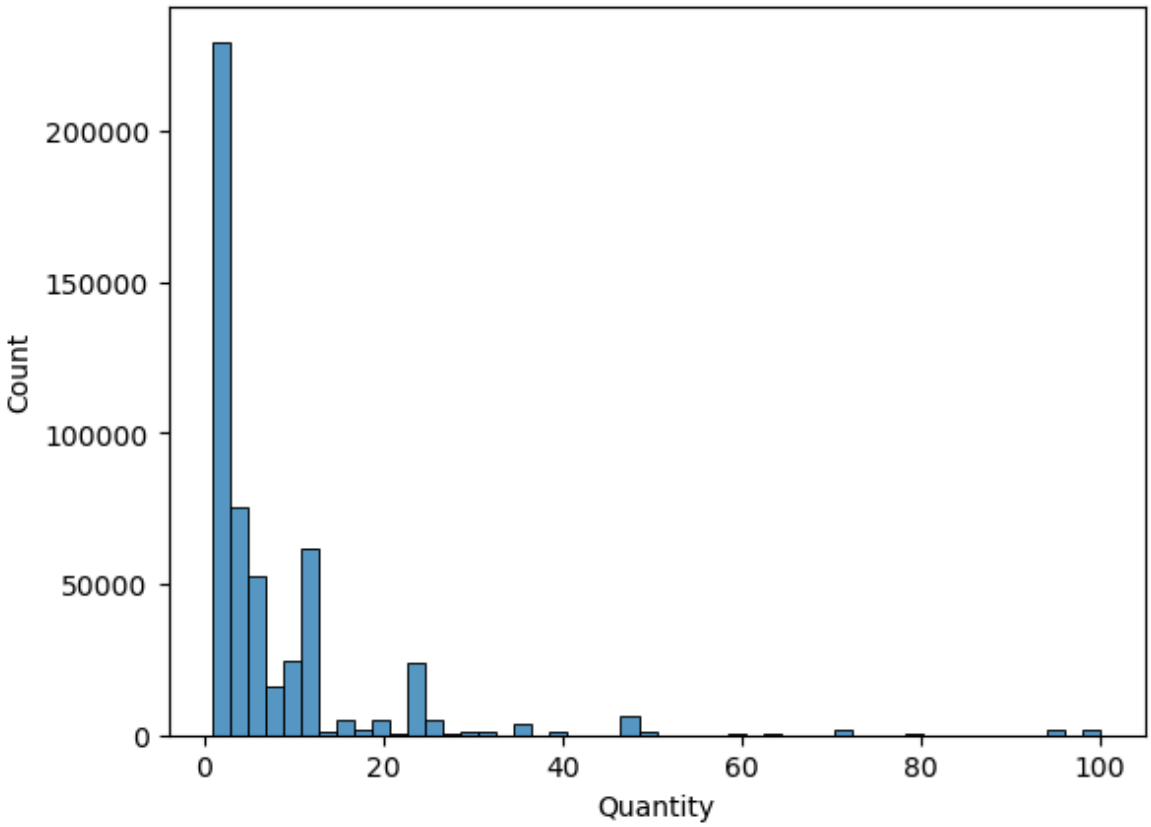
To better understand the variability and skewness in the transaction data, we examine the distributions of two key numerical features:

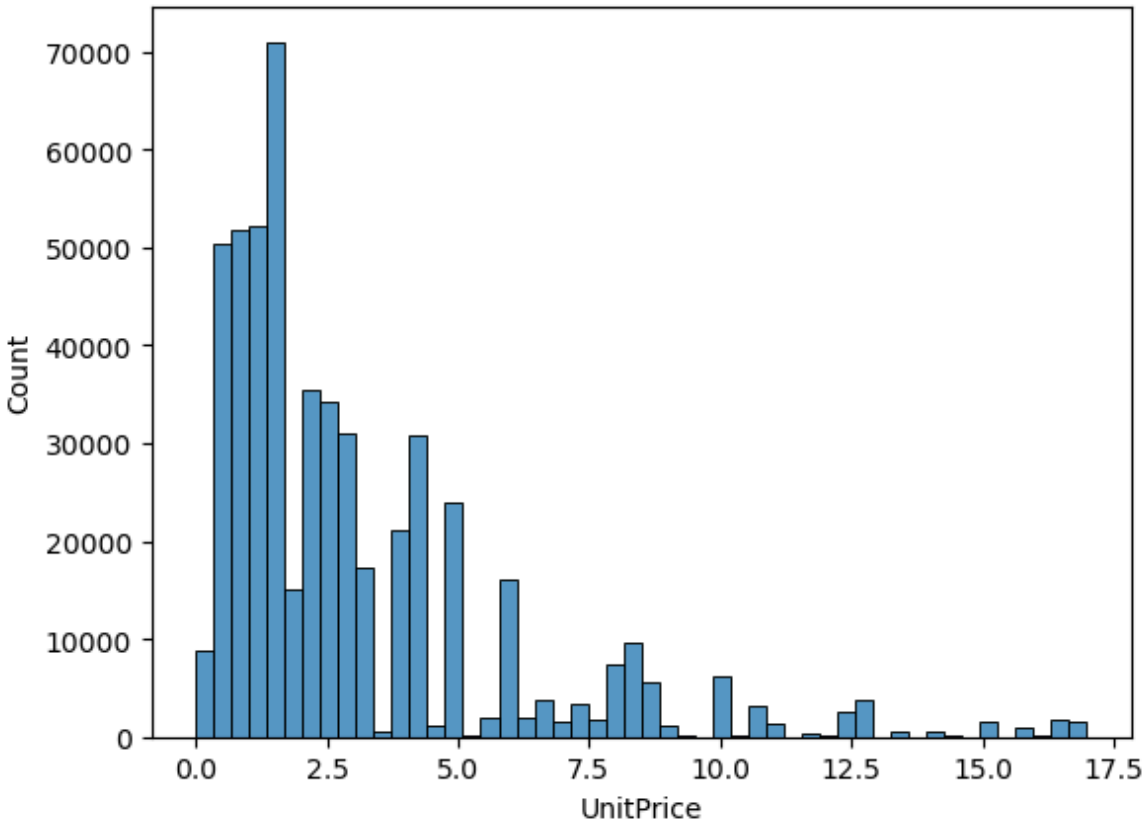
- `Quantity` : The number of units purchased per transaction.
- `UnitPrice` : The price per unit for each product.

Because both features contain extreme outliers that could distort the analysis, we focus on values below the 99th percentile for visualization.

These distributions provide insight into typical transaction sizes and prices, highlight unusual purchasing behavior, and help guide the construction of meaningful clusters in subsequent analyses.

In [13]:
```
# Some Distributions
df_clean = data_clean[
    (data_clean["Quantity"] > 0) &
    (data_clean["UnitPrice"] > 0)
]
q_upper = df_clean["Quantity"].quantile(0.99)
p_upper = df_clean["UnitPrice"].quantile(0.99)

sns.histplot(df_clean[df_clean["Quantity"] <= q_upper]["Quantity"], bins=50)
plt.show()

sns.histplot(df_clean[df_clean["UnitPrice"] <= p_upper]["UnitPrice"], bins=50)
plt.show()
```



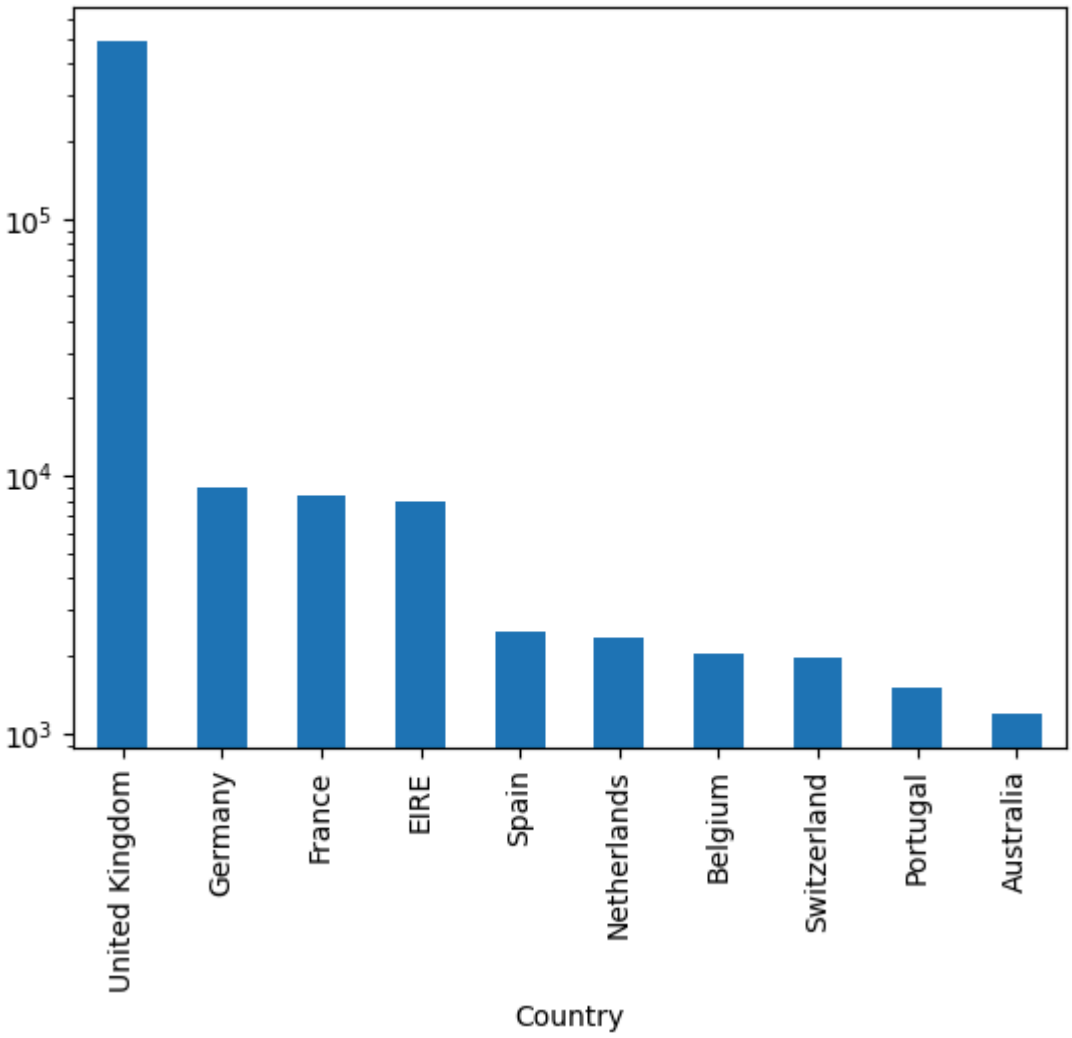


To examine the geographic distribution of transactions, we plot the top 10 countries by transaction count.

Because the UK dominates the dataset, we use a logarithmic scale on the y-axis to better visualize the relative frequency of other countries.

In [14]:
```
# Country Distribution (top 10)
data_clean["Country"].value_counts().head(10).plot(kind="bar")
```

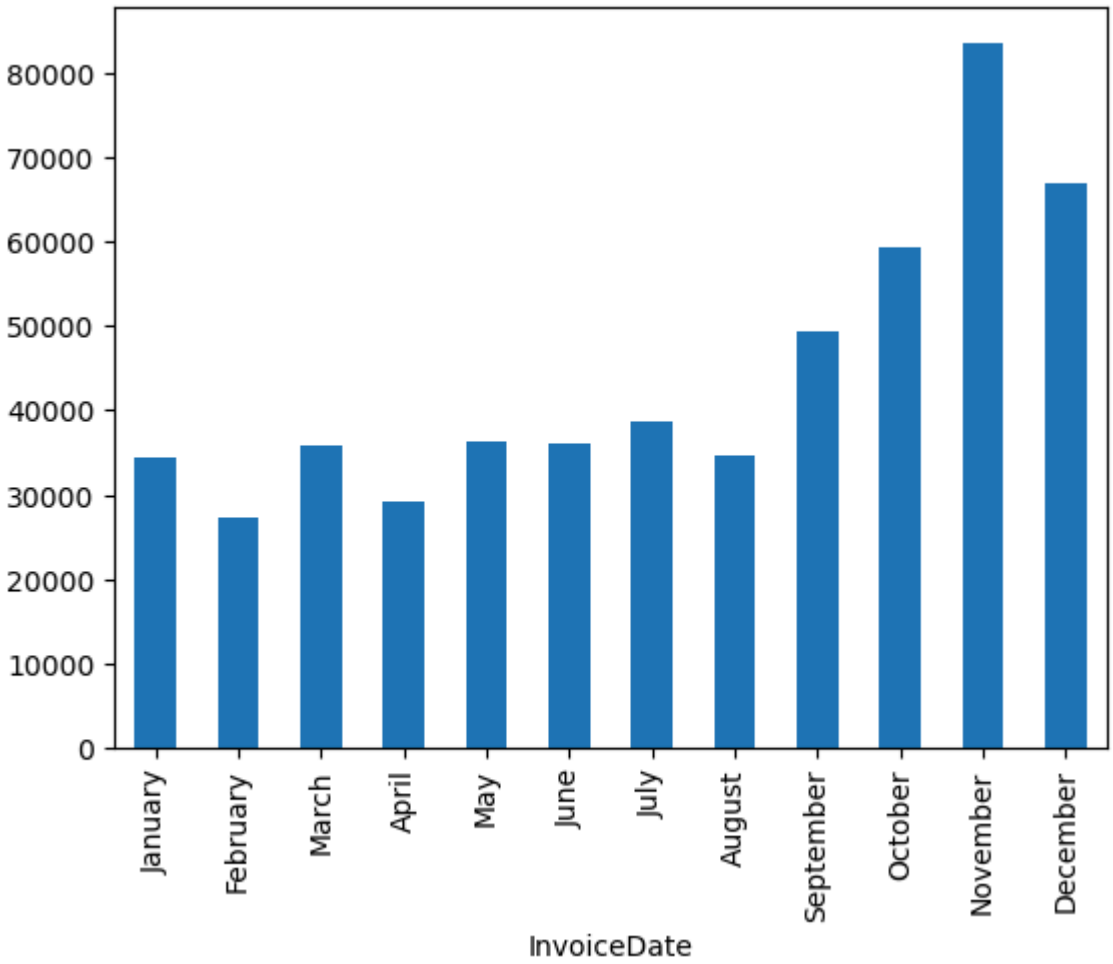
```
plt.yscale("log")
plt.show()
```



We can also examine the number of transactions per month to identify seasonal patterns or trends in purchasing behavior.

Understanding these temporal patterns can help in feature engineering - forexample incorporating month or season as a feature - and may reveal information that could influence clustering or customer segmentation.

In [15]:
```
# Distributions of data in regards to time
data_clean["InvoiceDate"] = pd.to_datetime(data_clean["InvoiceDate"])
data_clean["InvoiceDate"].dt.month_name().value_counts().reindex(["January", "Fe
plt.show()
```



3.5 Sparsity and Density

We assess the sparsity and density of the dataset by constructing a Customer–Product matrix, where each row represents a customer, each column represents a product, and the values correspond to the quantity purchased.

We calculate the density as the proportion of non-zero entries in the matrix, with sparsity defined as 1 minus the density. This analysis helps us understand how concentrated or dispersed purchases are across customers and products, which has implications for clustering and recommendation analyses.

In [16]:
```
# Customer-Product Matrix.
pivot = data_clean.pivot_table(index="CustomerID",
                               columns="StockCode",
                               values="Quantity",
                               aggfunc="sum",
                               fill_value=0)

# Density and Sparsity calculations
non_zero = np.count_nonzero(pivot)
total = pivot.size

density = non_zero / total
sparsity = 1 - density
```

```
print("Density:", density)
print("Sparsity:", sparsity)
```

```
Density: 0.016777430626190266
Sparsity: 0.9832225693738097
```

The Customer-product matrix has a **density of ≈ 1.68%** and a corresponding **sparsity of ≈ 98.32%**, meaning that each customer has purched only about 1-2% of all distinct stock codes in the catalogue.

Here we expected to see a high sparsity for retail data, as customers naturally consentrate on a small fraction of available products. For our invoice-level clustering approach, this sparsity is not a problem: we aggregate each invoice into a compact numerical feature vector before clustering, thereby avoiding the curse-of-dimensionality issues tgat arise from raw product-indicator vectors.

4. Feature Engineering

To prepare the data for clustering, we aggregate invoice-level features that capture key aspects of each transaction.
Each invoice will be represented as a single data point with the following features:

Transaction Value Features:

- Total revenue (`Quantity` × `UnitPrice`)
- Average unit price
- Maximum unit price
- Standard deviation of unit price

Basket Composition Features:

- Total quantity
- Number of unique items
- Average quantity per product

Cancellation Features:

- Binary cancellation indicator (whether the invoice was cancelled)

Time-based features:

- Month of invoice (to find seasonal trends etc.)

Geographical Features:

- Country of Customer

We also chose to flatten the column names for two reasons. First, it makes the dataset easier to work with when referencing individual features in later analysis steps. Second, it improves readability and clarity when reviewing the code and results throughout the report.

In [17]:
```
# Aggregate features per invoice
invoice_features = data_clean.groupby("InvoiceNo").agg({
    "UnitPrice": ["sum", "mean", "max", "std"],
    "Quantity": ["sum", "nunique", "mean"],
    "CustomerID": "first",
    "InvoiceDate": "first",
    "Country": "first"
})

# Flatten column names
invoice_features.columns = [
    "TotalPrice", "AvgPrice", "MaxPrice", "StdPrice",
    "TotalQuantity", "NumUniqueProducts", "AvgQuantityPerProduct",
    "CustomerID",
    "InvoiceDate",
    "Country"
]

# Invoice-level std can be NaN for one-line invoices; fill to keep downstream mo
invoice_features["StdPrice"] = invoice_features["StdPrice"].fillna(0.0)

# Cancellation indicator (kept for robustness/traceability)
invoice_features["Cancelled"] = invoice_features.index.str.startswith("C").astyp

invoice_features.head()
```

Out[17]:

InvoiceNo	TotalPrice	AvgPrice	MaxPrice	StdPrice	TotalQuantity	NumUniqueProducts
536365	27.37	3.910000	7.65	1.737316	40	
536366	3.70	1.850000	1.85	0.000000	12	
536367	58.24	4.853333	9.95	2.772929	83	
536368	19.10	4.775000	4.95	0.350000	15	
536369	5.95	5.950000	5.95	0.000000	3	

5. Vector-Space Analysis

To perform clustering, each invoice is represented in a vector in a multi-dimensional feature space. In order to select the columns needed for clustering analysis, we first do a correlation plot. This will help us identify highly correlated variables, which might affect clustering result and give wrong interpretations. Reasoning would be because one of the clustering methods we will use is Kmeans, which basically clusters by minimizing the SSE in the Euclidean distance. This means that the variables are expected to be independent in sorts. Therefore, highly correlated variables will be represented wrongly in the Euclidean distance, which might affect the clustering results.

We take do a correlation of numerical variables, i.e.:

- TotalPrice
- AvgPrice
- MaxPrice
- StdPrice
- TotalQuantity
- AvgQuantityPerProduct

`NumUniqueProducts` was excluded because it captures basket size in a way that overlaps substantially with `TotalQuantity`, introducing redundancy into the feature space. When included alongside features such as `AvgPrice` and `AvgQuantityPerProduct`, the silhouette scores became uniformly high across all values of k — a sign that clustering was being driven by feature redundancy rather than genuine structure.

In [18]:
```
invoice_corr = invoice_features[["TotalQuantity", "TotalPrice", "MaxPrice", "Std
invoice_corr
```

Out[18]:

	TotalQuantity	TotalPrice	MaxPrice	StdPrice	AvgPrice	AvgQ
TotalQuantity	1.000000	0.095504	0.029637	0.010574	-0.010211	
TotalPrice	0.095504	1.000000	0.805840	0.429235	0.467439	
MaxPrice	0.029637	0.805840	1.000000	0.389798	0.848737	
StdPrice	0.010574	0.429235	0.389798	1.000000	0.149054	
AvgPrice	-0.010211	0.467439	0.848737	0.149054	1.000000	
AvgQuantityPerProduct	0.832836	-0.012182	-0.005420	-0.007392	-0.001872	

We notice the following:

- `TotalPrice` and `TotalQuantity` have low correlation between them (approximately 1%).
- `MaxPrice`, `StdPrice`, `AvgPrice` have relatively high correlation with `TotalPrice` from 50%.
- `AvgQuantityPerProduce` have high correlation with `TotalQuantity`.

Accordingly, we choose to select the following features:

- `TotalPrice:` total value of the invoice
- `TotalQuantity:` total number of items purchased

The data clustered includes only transactions that are not cancelled, because we are currently interested in the patterns from actual transactions.

Clustering these features hopefully gives us an ability to segment the customers based on their purchase and where they are from.

Along with the features above, the following variable will be used in interpreting the results.

- `Country:` The country of the customer

We have not used `CustomerID`, since it contains missing values, and will not help a lot with interpreting the results.

Before clustering, the features are normalized to ensure that differences in scale do not dominate distance computations.

5.1 Vector Representation

In order to construct the vector we do the following:

1. Select transaction-value features (`TotalPrice`, `TotalQuantity`) for clustering.
2. Keep only valid completed transactions (`TotalPrice > 0`) and trim extreme outliers for stability and to get better clustering results of the main interactions.
3. Encode country for later interpretation.
4. Normalize `TotalPrice` and `TotalQuantity` with `RobustScaler` due to skewed distributions.
5. Build a final clustering matrix with normalized numeric features only.

In [20]:
```
# Construct vector representation
X = invoice_features[["TotalPrice", "TotalQuantity", "Country"]].copy()

# Keep valid completed transactions for clustering workflow
X = X.query("TotalPrice > 0")
X
```

Out[20]:

InvoiceNo	TotalPrice	TotalQuantity	Country
536365	27.37	40	United Kingdom
536366	3.70	12	United Kingdom
536367	58.24	83	United Kingdom
536368	19.10	15	United Kingdom
536369	5.95	3	United Kingdom
...	...	...	...
581583	3.30	76	United Kingdom
581584	2.57	120	United Kingdom
581585	37.78	278	United Kingdom
581586	20.23	66	United Kingdom
581587	44.50	105	France

19959 rows × 3 columns

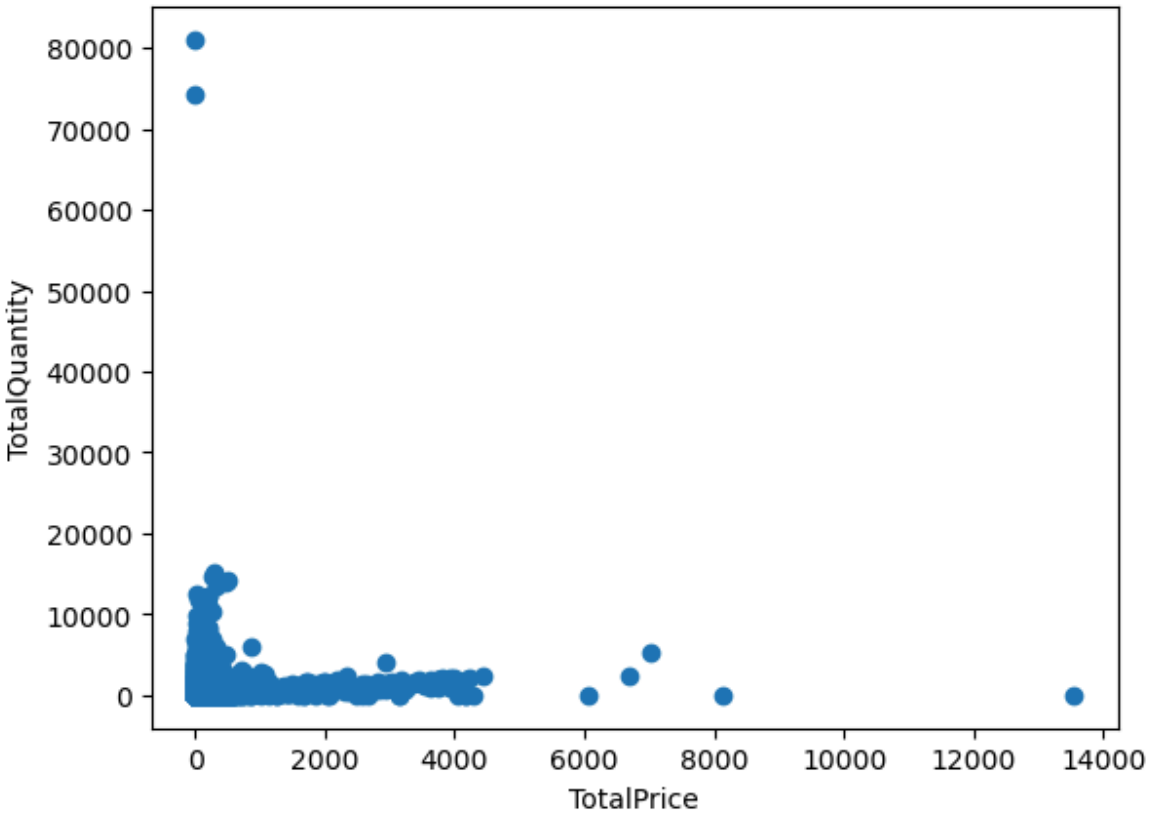
In [21]:
```
# Encode country for visualization only
unique_countries = X["Country"].unique()

le = LabelEncoder()
le.fit(unique_countries)

X["CountryEncoded"] = le.transform(X["Country"])
X[["TotalPrice", "TotalQuantity", "Country", "CountryEncoded"]].head()
```

Out[21]:

InvoiceNo	TotalPrice	TotalQuantity	Country	CountryEncoded
536365	27.37	40	United Kingdom	36
536366	3.70	12	United Kingdom	36
536367	58.24	83	United Kingdom	36
536368	19.10	15	United Kingdom	36
536369	5.95	3	United Kingdom	36

We visualize `TotalPrice` and `TotalQuantity`.

In [22]:
```
plt.scatter(X["TotalPrice"], X["TotalQuantity"])
plt.xlabel('TotalPrice')
plt.ylabel('TotalQuantity')
plt.show()
```



In [23]:
```
X[["TotalPrice", "TotalQuantity"]].describe()
```

Out[23]:

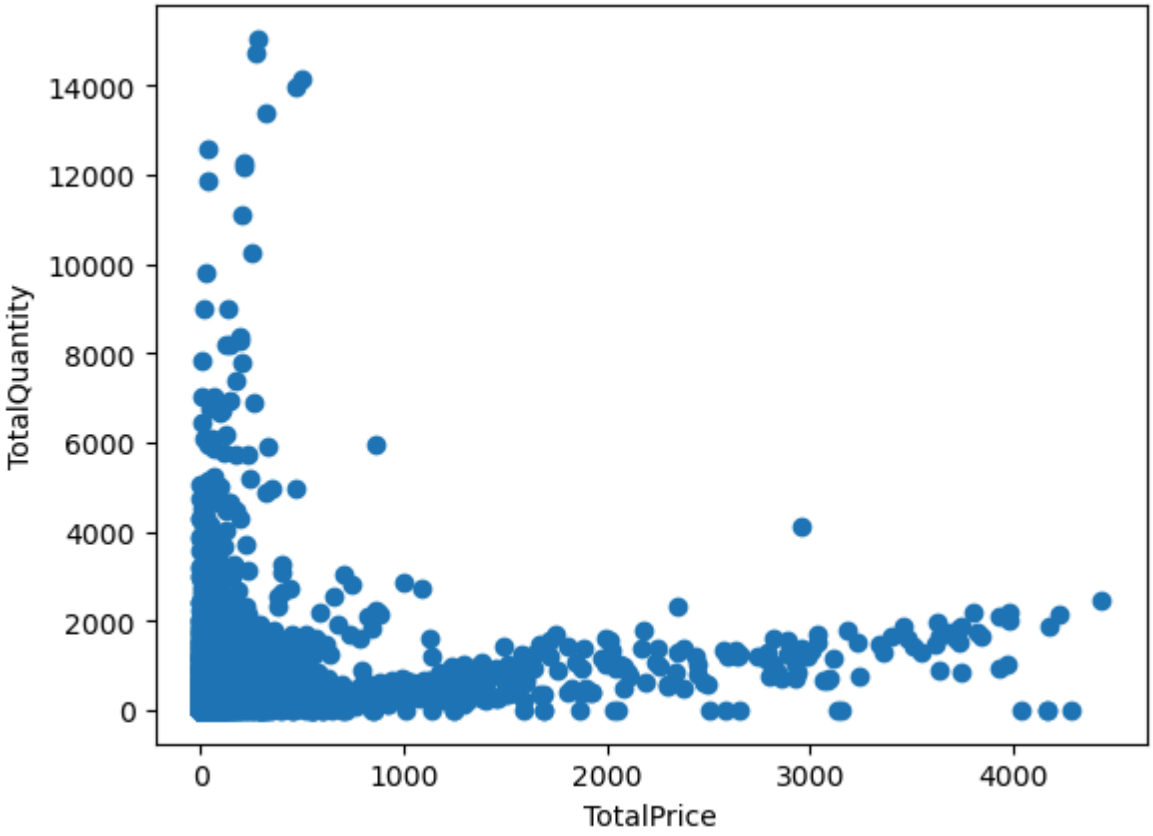
	TotalPrice	TotalQuantity
count	19959.000000	19959.000000
mean	103.230909	280.094093
std	302.128556	955.373491
min	0.060000	1.000000
25%	18.335000	70.000000
50%	45.420000	151.000000
75%	90.175000	296.000000
max	13541.330000	80995.000000

We notice that `TotalPrice` and `TotalQuantity` includes extreme outliers, which is squishing any possible patterns that could be used in clustering. Furthermore, When looking at the description of X, we notice that `TotalPrice` has minimum value equal to zero, which indicates that some of the cancelled transactions were hidden as having a price of zero and not removed.

For now we will remove the outliers, as well as any hidden transactions. Therefore we will only include:

1. `TotalPrice` bigger than zero, which should eliminate any cancelled transactions.
2. `TotalPrice` less than 5000 and `Total Quantity` less than 20000, which should remove any extreme values.

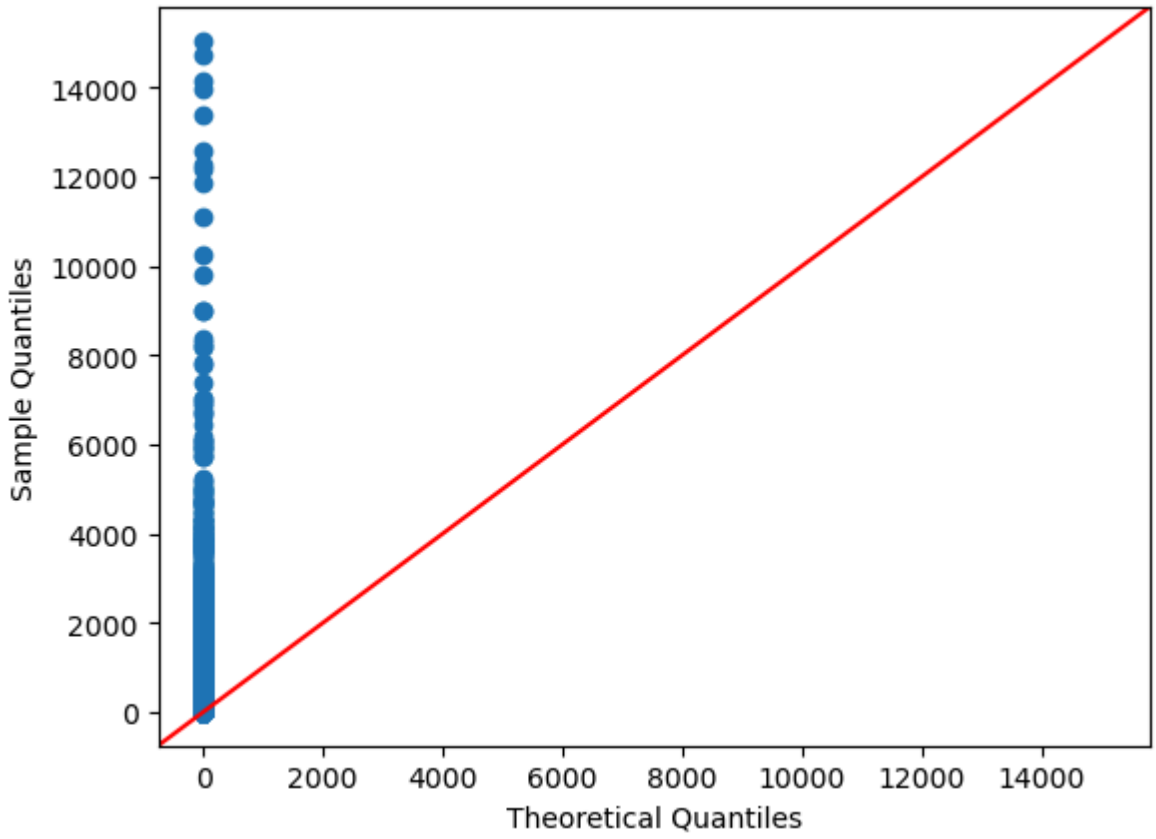
In [24]:
```
X = X.query('TotalPrice < 5000 & TotalQuantity < 20000 & TotalPrice > 0')

plt.scatter(X["TotalPrice"], X["TotalQuantity"])
plt.xlabel('TotalPrice')
plt.ylabel('TotalQuantity')
plt.show()
```



After normalization, most transactions are concentrated near the origin because the majority of invoices contain small quantities and relatively low total value. No clear clustering is apparent in the visualization.

We now normalize `TotalPrice` and `TotalQuantity`. Before doing that, we check the distribution of the data to see if we can assume it is normally distributed.

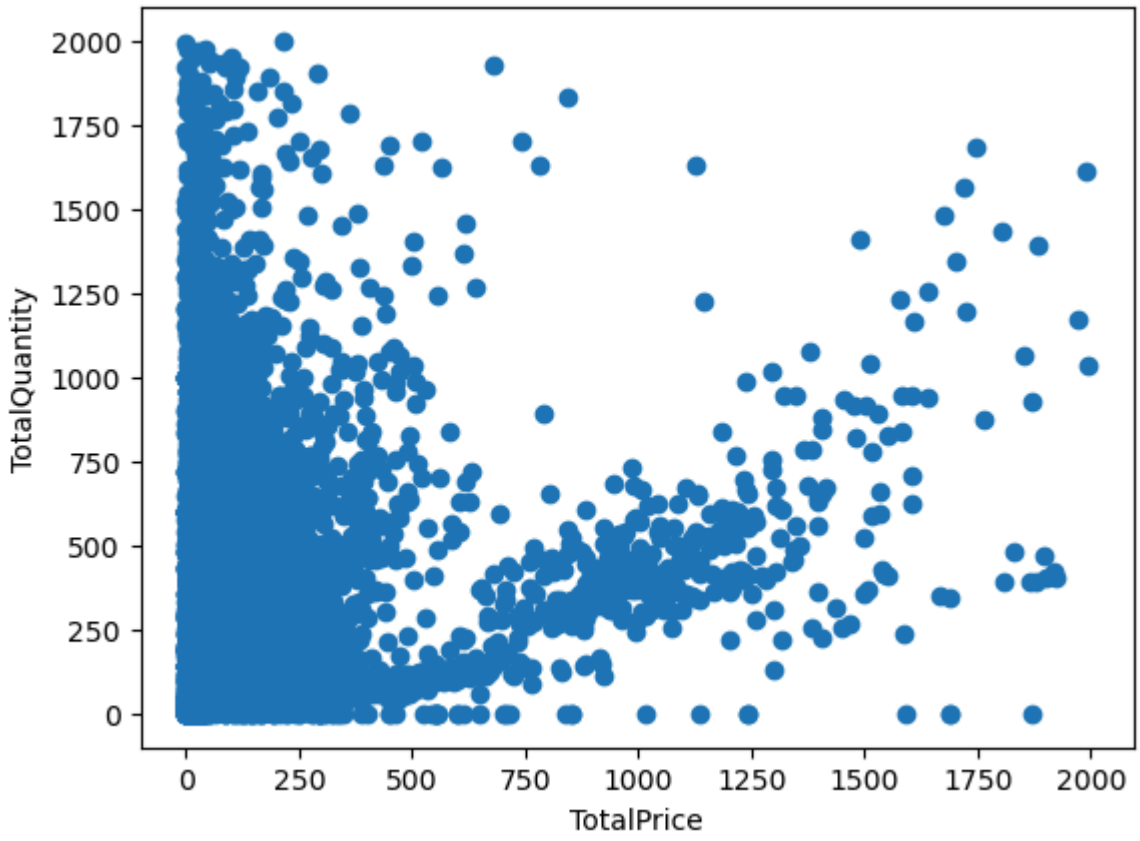
In [25]:
```
fig = sm.qqplot(X[['TotalPrice', 'TotalQuantity']], line='45')
plt.show()
```



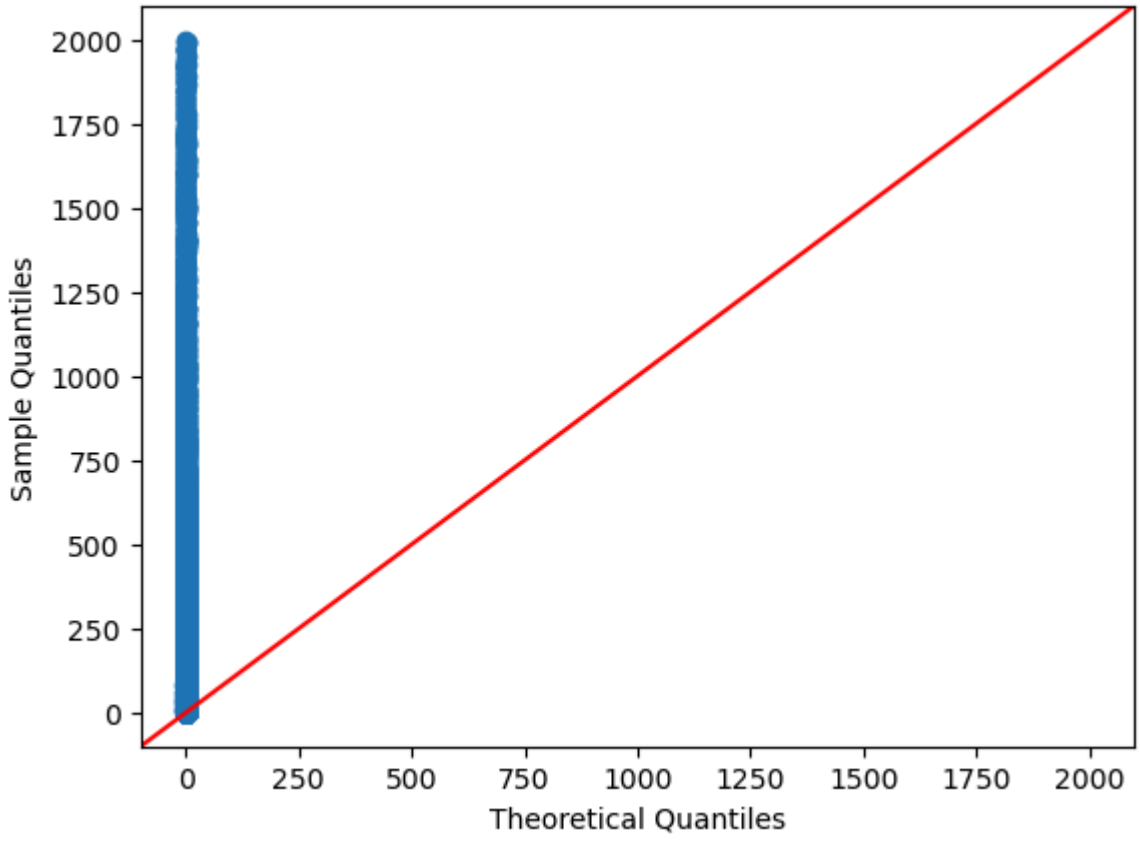
We see that TotalPrice and TotalQuantity is heavily skewed. Therefore we cannot assume it follows a normal distribution.

Most observations lie below 2000 for both TotalPrice and TotalQuantity, while a small number of extreme transactions create a long right tail. We try to visualize the data where both TotalPrice and TotalQuantity is less than 2000.

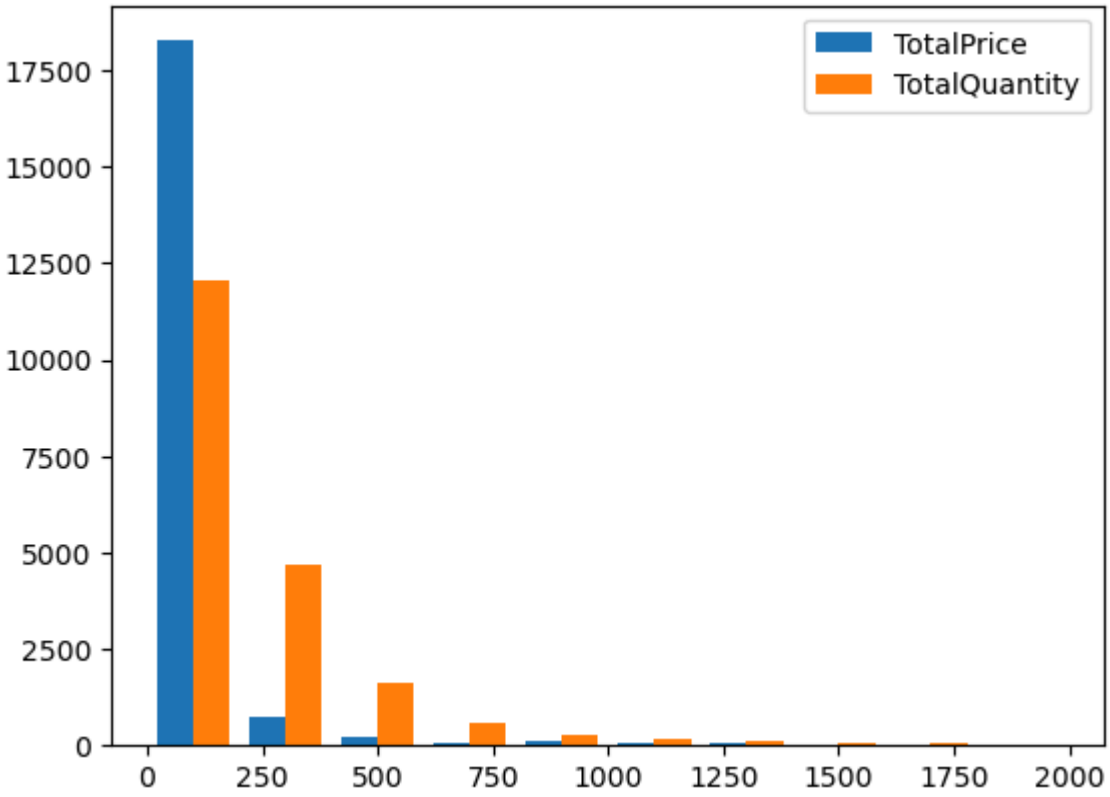
In [26]:
```
X_test = X.copy()
X_test= X_test.query('TotalPrice < 2000 & TotalQuantity < 2000')

plt.scatter(X_test["TotalPrice"], X_test["TotalQuantity"])
plt.xlabel('TotalPrice')
plt.ylabel('TotalQuantity')
plt.show()
```



In [27]:
```
fig = sm.qqplot(X_test[['TotalPrice', 'TotalQuantity']], line='45')
plt.show()
```



In [28]:
```
plt.hist(X_test[['TotalPrice', 'TotalQuantity']], label=['TotalPrice', 'TotalQua
plt.legend(loc="upper right")
plt.show()
```



We see that points > 2000 squished interesting trends in both TotalPrice and TotalQuantity, when we excluded these points, we were able to clearly see them.

When we did a histogram and a qqplot of the new data, we see that it is still skewd. However, the histogram is now more spread out than it was previously.

We use this new filtered data as our basis for the clustering. Since the data is not normalized, we use a package from sklearn called RobustScaler, which scales the data according to their quantile range.

In [29]:
```
X = X_test
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X[["TotalPrice", "TotalQuantity"]])
X_scaled
```

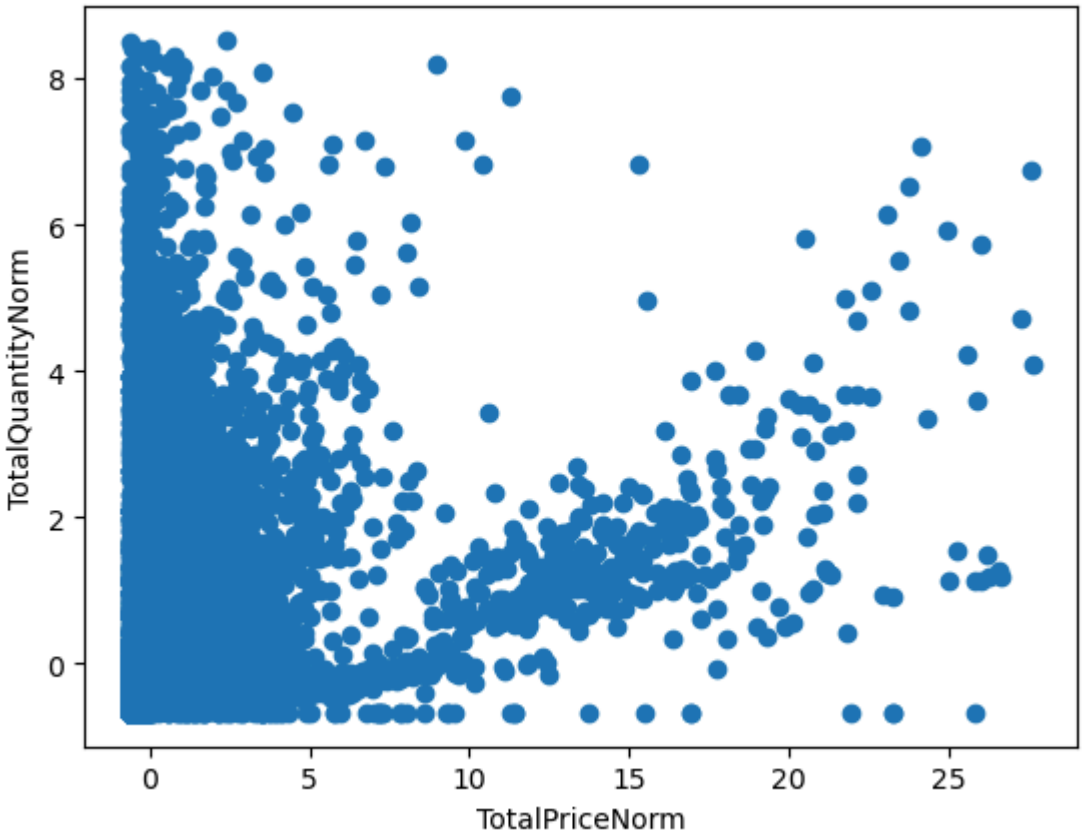
Out[29]:
```
array([[-0.25021252, -0.49769585],
       [-0.58557665, -0.62672811],
       [ 0.1871635 , -0.29953917],
       ...,
       [-0.10272032,  0.59907834],
       [-0.35137433, -0.37788018],
       [-0.00750921, -0.19815668]], shape=(19626, 2))
```

In [30]:
```
X["TotalPriceNorm"] = X_scaled[:, 0]
X["TotalQuantityNorm"] = X_scaled[:, 1]

# Final matrix used by clustering models
X_cluster = X[["TotalPriceNorm", "TotalQuantityNorm"]].copy()
X_cluster.describe()
```

Out[30]:

	TotalPriceNorm	TotalQuantityNorm
count	19626.000000	19626.000000
mean	0.579919	0.347533
std	2.357097	1.155803
min	-0.637149	-0.677419
25%	-0.380596	-0.364055
50%	0.000000	0.000000
75%	0.619404	0.635945
max	27.620714	8.525346

In [31]:
```
plt.scatter(X_cluster["TotalPriceNorm"], X_cluster["TotalQuantityNorm"])
plt.xlabel("TotalPriceNorm")
plt.ylabel("TotalQuantityNorm")
plt.show()
```



6 Clustering method

Since no apparent clustering is seen, we experiment with clustering of different types. And so we try out:

1. **kmeans clustering**: We use `k-means++` initialization, which improves the selection of initial centroids and generally leads to more stable clustering results. We experiment with k = 2, ..., 10 then choose the best k based on both the elbow score and the silhuoette score

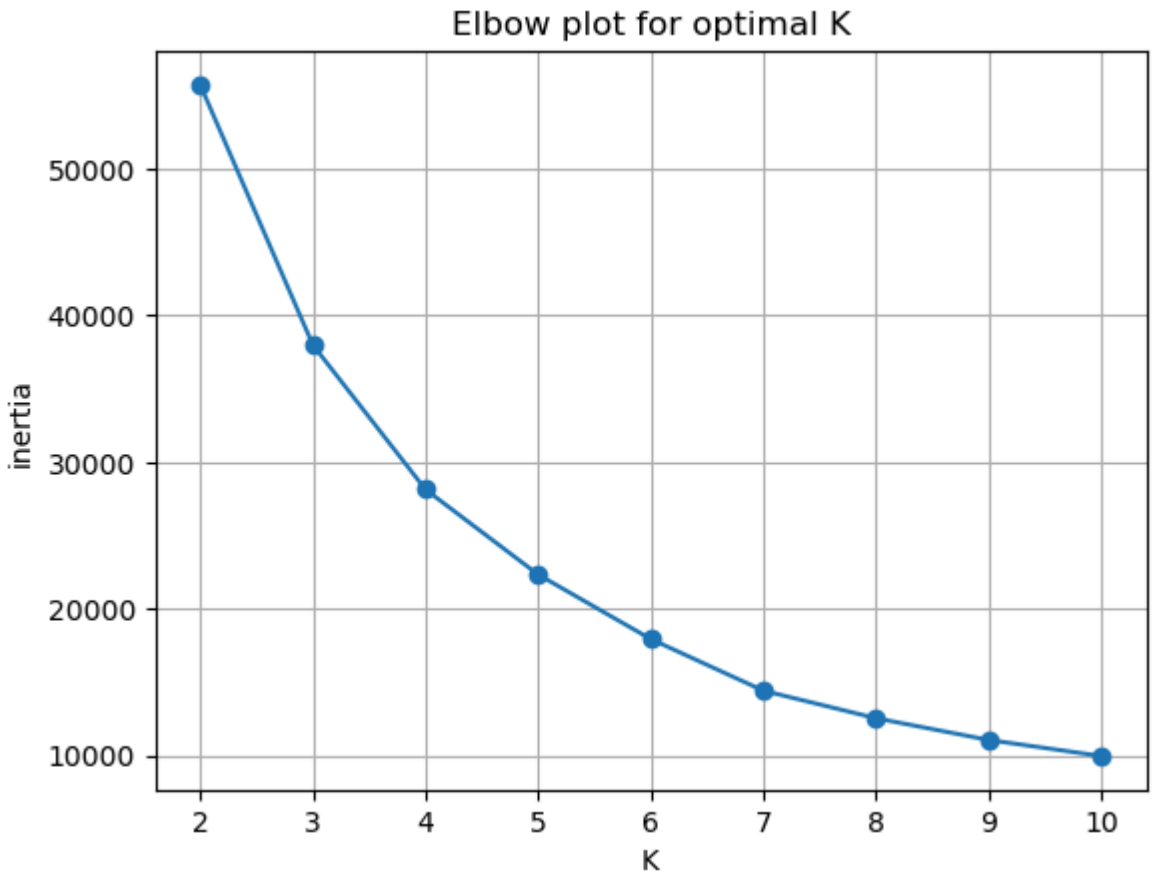
P.S.: We start from k = 2, as silhouette measures separation between clusters, which requires at least 2 clusters.

2. **Hierarchical clustering**: We plot a dendogram to determine the best partition. We then parition based on that.

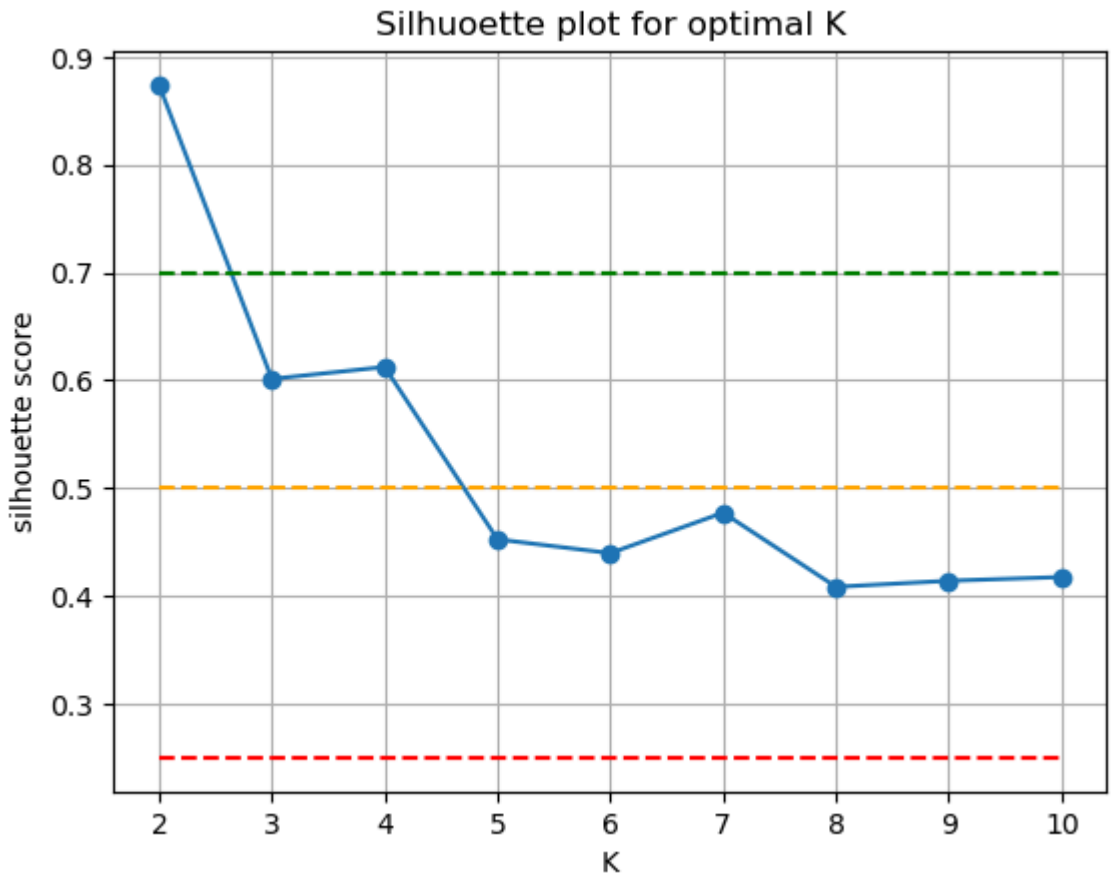
6.1 Kmeans

In [32]:
```
elbow_score = []
silhouette_scores = []
kmeans_models = {}

K = range(2, 11)

for k in K:
    model = KMeans(
        n_clusters=k,
        init="k-means++",
        random_state=RANDOM_SEED,
        n_init=20
    )
    labels_k = model.fit_predict(X_cluster)
    kmeans_models[k] = model

    elbow_score.append(model.inertia_)
    silhouette_scores.append(silhouette_score(X_cluster, labels_k))
```

In [33]:
```
plt.plot(K, elbow_score, '-o')
plt.title('Elbow plot for optimal K')
plt.xlabel('K')
plt.ylabel('inertia')
plt.xticks(K)
plt.grid()
plt.show()
```



In [34]:
```
plt.plot(K, silhouette_scores, '-o')
plt.hlines(y=[0.7, 0.5, 0.25],xmin=K[0], xmax=K[-1],colors=['green', 'orange', '
plt.title('Silhuoette plot for optimal K')
plt.xlabel('K')
plt.ylabel('silhouette score')
plt.grid()
plt.show()
```



In [35]:
```
for k in K:
    print(f'Avg silhouette score for k = {k} is {silhouette_scores[k-2]}')
```

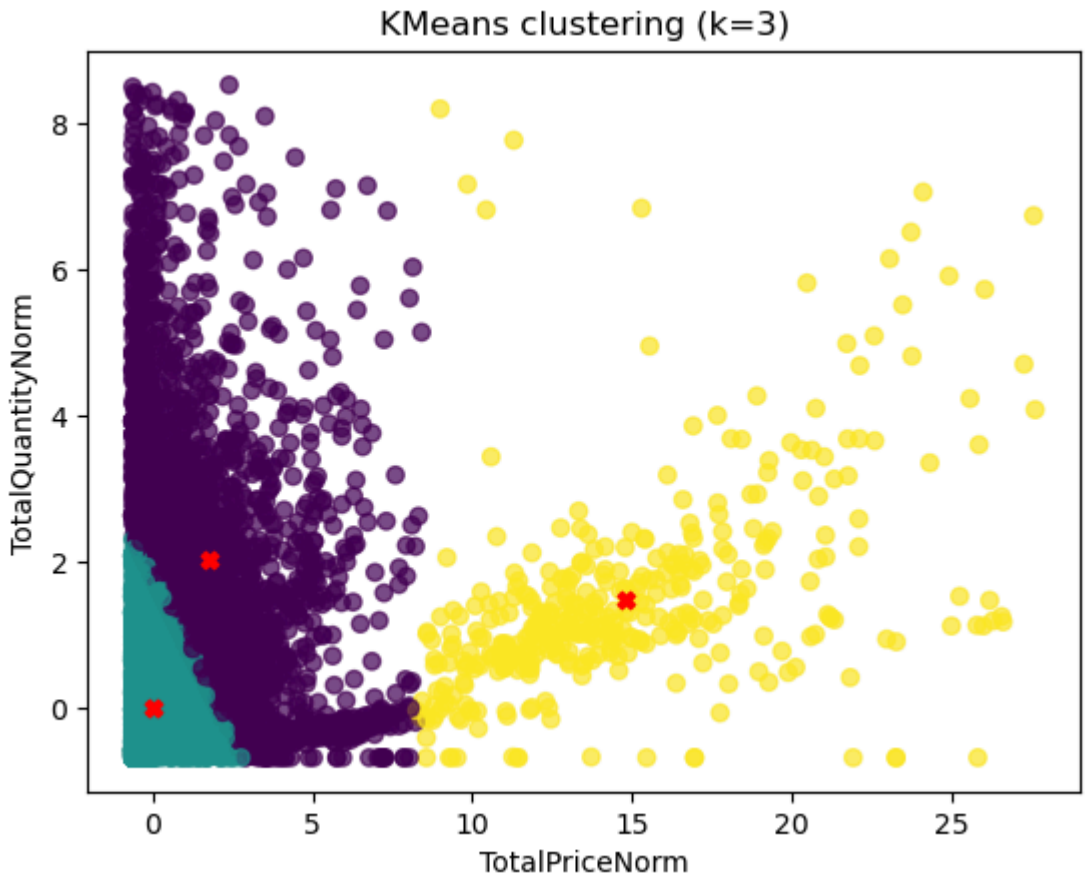
```
Avg silhouette score for k = 2 is 0.8739256377966855
Avg silhouette score for k = 3 is 0.6012937269222141
Avg silhouette score for k = 4 is 0.6123902692158902
Avg silhouette score for k = 5 is 0.45247886399410614
Avg silhouette score for k = 6 is 0.4396949203510318
Avg silhouette score for k = 7 is 0.47720434952259577
Avg silhouette score for k = 8 is 0.4084429149521572
Avg silhouette score for k = 9 is 0.4140580755995856
Avg silhouette score for k = 10 is 0.4175172965144722
```

From the elbow plot, it seems that k=3 to k=6 are sort of the elbow clusters.

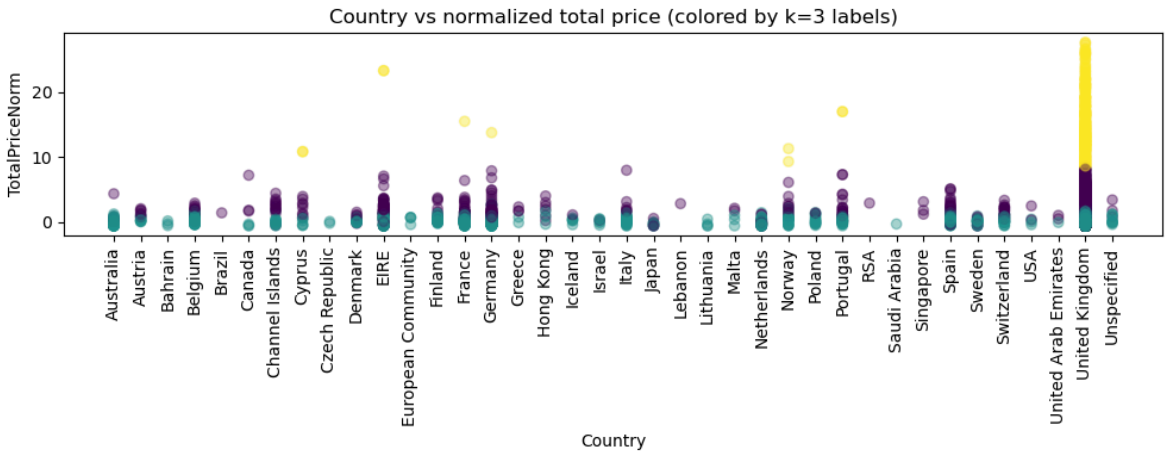
The silhouette plot tells us that of the these clusters k=3 is the strongest.

We visualize k=3 to see the clustering outcome

In [36]:
```
best_k = 3
model_k3 = kmeans_models[best_k]
labels = model_k3.labels_
centroids = model_k3.cluster_centers_

plt.scatter(X_cluster["TotalPriceNorm"], X_cluster["TotalQuantityNorm"], c=label
plt.scatter(centroids[:, 0], centroids[:, 1], marker="X", color="red")
plt.xlabel("TotalPriceNorm")
plt.ylabel("TotalQuantityNorm")
plt.title(f"KMeans clustering (k={best_k})")
plt.show()
```



In [37]:
```
country_codes = sorted(X["CountryEncoded"].unique())
country_names = le.inverse_transform(country_codes)

plt.figure(figsize=(10, 4))
plt.scatter(X["CountryEncoded"], X["TotalPriceNorm"], c=labels, alpha=0.4)
plt.xticks(country_codes, country_names, rotation=90)
plt.xlabel("Country")
plt.ylabel("TotalPriceNorm")
plt.title("Country vs normalized total price (colored by k=3 labels)")
plt.tight_layout()
plt.show()
```



According to the plots, the clusters segments the transactions into following:

1. **Yellow Cluster**: Customers mainly from the UK with most overall purchase (They bought in total 550 dollar and more and in quantities 150 items in total and more).

P.S.: price and quantities is guessed from comparing the clustered plot with the plot without normalization

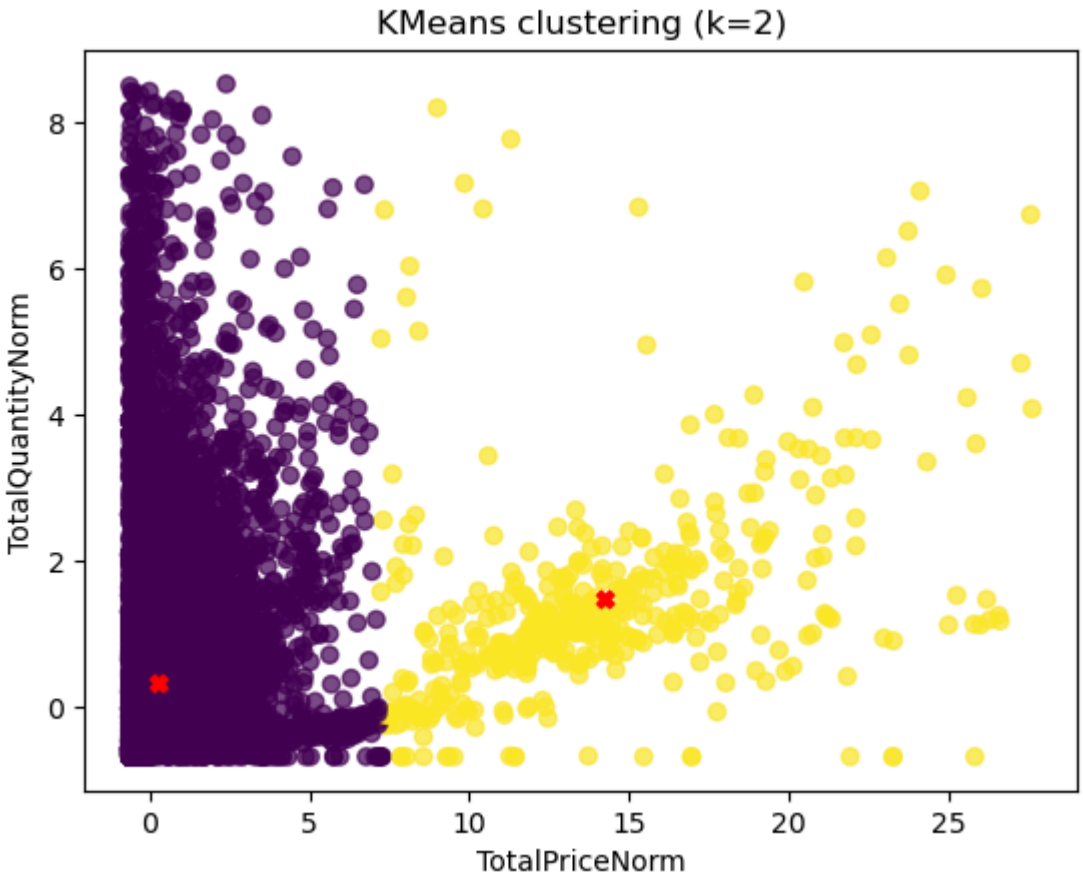
2. **Blue Cluster**: Customers with least purchase (They bought at most 500 items with less than 250 dollar). These types of customers are scattered in all countries.

3. **Violet Clusters**: Customers who bought in total quantity and price more than the blue cluster but lesser than the violet cluster.

Other than that, we notice some countries having very little data, like brazil and Lebanon. We also notice that transactions from countrues other than UK very rarely belong to the Yellow Cluster, which shows that focusing on UK customers is very important for the business. If the business chooses to become more international, they might start with countries that has transactions belonging to the yellow cluster like Norway and EIRE.

In hierarchical clustering, it chooses two clusters. To compare it to kmeans, we look at when $k = 2$

In [38]:
```
model_k2 = kmeans_models[2]
labels_2 = model_k2.labels_
centroids = model_k2.cluster_centers_

plt.scatter(X_cluster["TotalPriceNorm"], X_cluster["TotalQuantityNorm"], c=label
plt.scatter(centroids[:, 0], centroids[:, 1], marker="X", color="red")
plt.xlabel("TotalPriceNorm")
plt.ylabel("TotalQuantityNorm")
plt.title("KMeans clustering (k=2)")
plt.show()
```



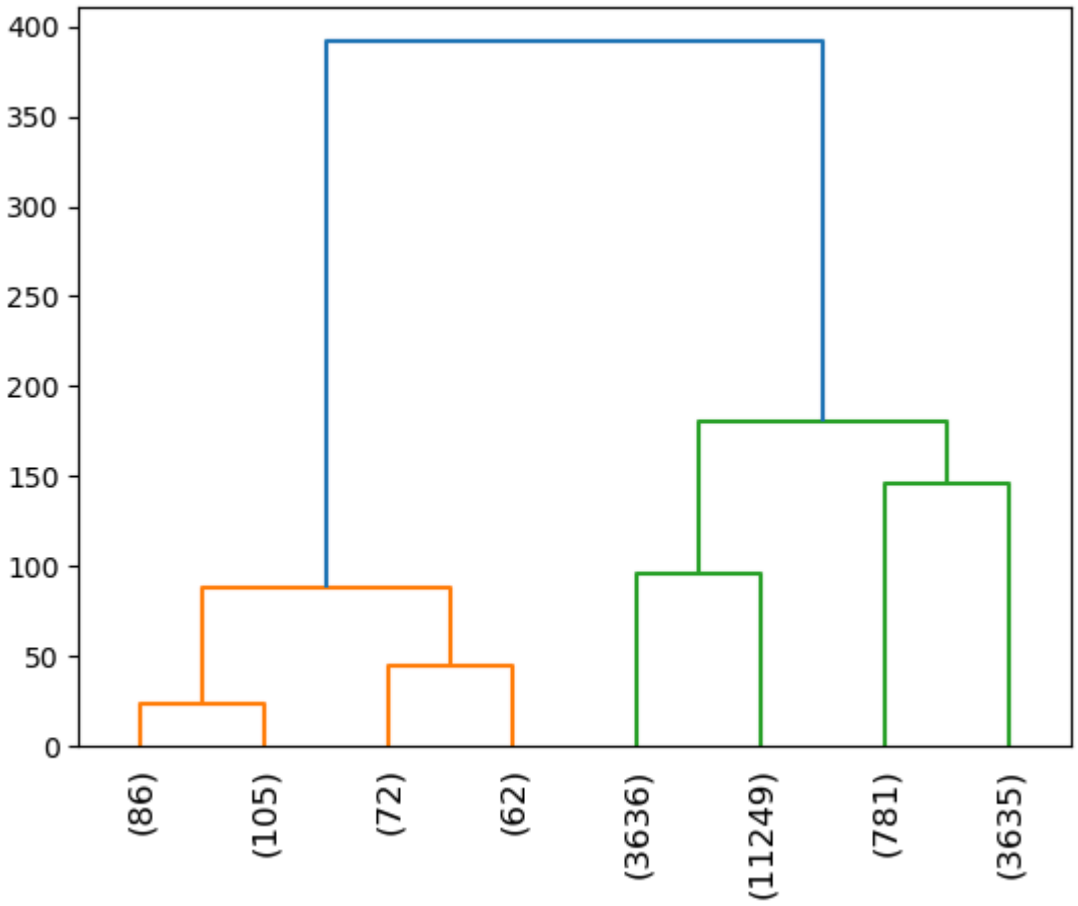
`Kmeans` for $k = 2$ merged the violet and blue cluster together. Otherwise, we have approximately the same type of cluster.

6.2 Agglomerative clustering

We try to cluster the algorithm using Agglomerative clustering. We change the method to 'ward', which minimizes distance based on the Ward variance minimization algorithm.

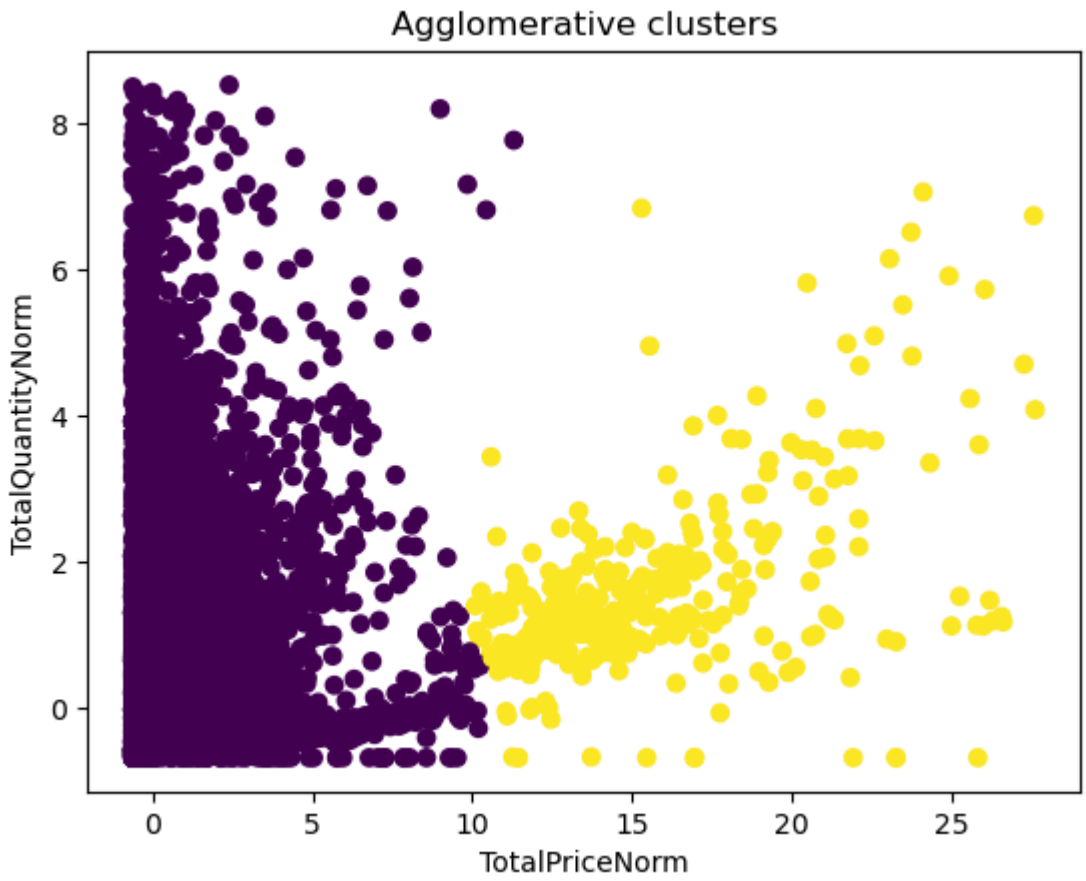
We first plot the dendogram to see distances of points. Based on the maxmimum distances we choose n_clusters for the sklearn AgglomerativeClustering algorithm

In [39]:
```
dendrogram = sch.dendrogram(
    sch.linkage(X_cluster, method="ward"),
    leaf_rotation=90,
    truncate_mode="level",
    p=2
)
```



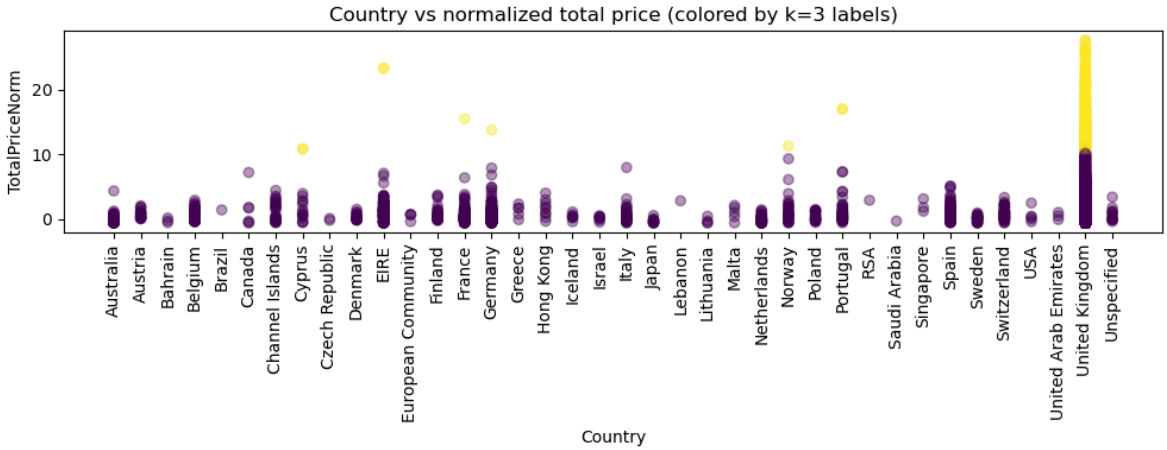
In [40]:
```
hc = AgglomerativeClustering(n_clusters=2, linkage="ward")
labels_hc = hc.fit_predict(X_cluster)
```

In [41]:
```
plt.scatter(X_cluster["TotalPriceNorm"], X_cluster["TotalQuantityNorm"], c=label
plt.title("Agglomerative clusters")
plt.xlabel("TotalPriceNorm")
plt.ylabel("TotalQuantityNorm")
plt.show()
```



Heirarchical clustering detects two main types of clustering. A cluster approximately > 750 of the `TotalPrice` and one lesser than 750 of the `TotalPrice`. Like previously, the number 750 is taken by comparing this plot to the original plot before normalizing `TotalPrice` and `Total Quantity`

In [42]:
```
country_codes = sorted(X["CountryEncoded"].unique())
country_names = le.inverse_transform(country_codes)

plt.figure(figsize=(10, 4))
plt.scatter(X["CountryEncoded"], X["TotalPriceNorm"], c=labels_hc, alpha=0.4)
plt.xticks(country_codes, country_names, rotation=90)
plt.xlabel("Country")
plt.ylabel("TotalPriceNorm")
plt.title("Country vs normalized total price (colored by k=3 labels)")
plt.tight_layout()
plt.show()
```



The points classified differently between Hierarchical and kmeans clustering mainly comes from countries other than UK. We see for example that Norway here only has one transaction belonging to the yellow cluster.

6.3 Conclusion

Overall. Heirarchical clustering seems to have a better prediction of the two clusters. It was able to correctly classify out the outliers in the violet cluster.

Of course it makes sense that heirarchical clustering has done better than kmeans, since the clusters themselves are weirdly shaped and don't seem convex. If given time, we might choose to experiment DBSCAN to see the results when clustering is based on density.

We also must state that choosing `ward` in heirarchical clustering seemed to have given it the ability to predict better the outliers in the two clusters, because it takes the distance based on the variance. Therefore, choosing `ward` seemed like a pretty good choice for this type of cluster..

7. Anomaly Detection

To identify extreme transactions, we apply both **univariate** and **multivariate** methods:

1. **Z-score Method** - flags invoices that are extreme for a single feature,forexample very high TotalPrice.
2. **Isolation Forest** - identifies invoices that are unusual across multiple features simultaneously.

The Z-score method was selected as a baseline approach for anomaly detection due to its simplicity and strong grounding in staitstical theory. It measures how far a given observation deviates from the mean in terms of standard deviation.
Since Z-score rely on assumptions about the data distributions and may not capture all types of anomalies, we were motivated to use a complimentary approach such as Isolation Forest.

Isolation Forest was chosen as a complementary anomaly detection method due to its ability to detect anomalies in a multivariate setting. Unlike the Z-score method, which evaluates each feature independently, Isolation Forest considers combinations of features and can therefore identify more complex and subtle anomalies.
Additionally, Isolation Forest does not assume any specific data distribution, making it well suited for this dataset, which contains skewed and diversified features such as transaction values and quantities.

Using two different anomaly detection methods provide a more robust analysis. The Z-score method captures extreme values in individual varibles, while Isolation Forest detects anomalies based on combinations of multiple features.
This dual approach allows us to capture both obvious outliers and more subtle

multivariate anomalies, and allow us to compare their results to better understand the structure of the data.

7.1 Z-Score Method

The Z-score method identifies invoices that are extreme in **a single feature**.
Here, we compute the Z-score for `TotalPrice` and `TotalQuantity` for each invoice. These features were chosen because they are most relevant in regards to our objective of identifying high-value and unusually large transactions.

A threshold of $|z| > 3$ was used to identify anomalies of our invoices. This choice is based on the empirical rule for normally distributed data, which states that approximately 99.7 of observations lie within $\pm$ 3 standard deviations. Observations outside this range are therefore statistically rare and are commonly considered outliers in practice.

While the data is not strictly normally distributed, this threshold provides a well established and interpretable baseline for detecting **obviously unusual** values in individual features compared to the rest of the dataset.

In [43]:
```
# Select features for univariate anomaly detection (completed invoices only)
X_anom = invoice_features.loc[
    invoice_features["Cancelled"] == 0,
    ["TotalPrice", "TotalQuantity"]
].copy()

# Compute z-scores and guard against constant-column NaNs
z_scores = X_anom.apply(zscore).fillna(0.0)

# Initialize flags
invoice_features["Z_TotalPrice"] = np.nan
invoice_features["Z_TotalQuantity"] = np.nan
invoice_features["PriceAnomaly"] = 0
invoice_features["QuantityAnomaly"] = 0
invoice_features["ZScoreAnomaly"] = 0

# Store z-scores
invoice_features.loc[X_anom.index, "Z_TotalPrice"] = z_scores["TotalPrice"]
invoice_features.loc[X_anom.index, "Z_TotalQuantity"] = z_scores["TotalQuantity"]

# Flag anomalies per feature
invoice_features.loc[X_anom.index, "PriceAnomaly"] = (
    abs(invoice_features.loc[X_anom.index, "Z_TotalPrice"]) > 3
).astype(int)
invoice_features.loc[X_anom.index, "QuantityAnomaly"] = (
    abs(invoice_features.loc[X_anom.index, "Z_TotalQuantity"]) > 3
).astype(int)

# Overall anomaly flag
invoice_features.loc[X_anom.index, "ZScoreAnomaly"] = (
    (invoice_features.loc[X_anom.index, "PriceAnomaly"] == 1) |
    (invoice_features.loc[X_anom.index, "QuantityAnomaly"] == 1)
).astype(int)

# Print summary
```

```
print("Price anomalies:", invoice_features["PriceAnomaly"].sum())
print("Quantity anomalies:", invoice_features["QuantityAnomaly"].sum())
print("Total Z-score anomalies:", invoice_features["ZScoreAnomaly"].sum())

cols_to_show = [
    "TotalPrice","TotalQuantity","Z_TotalPrice","Z_TotalQuantity","ZScoreAnomaly
]

print("\nExample price anomalies:")
print(invoice_features.loc[invoice_features["PriceAnomaly"] == 1, cols_to_show].

print("\nExample quantity anomalies:")
print(invoice_features.loc[invoice_features["QuantityAnomaly"] == 1, cols_to_sho
```

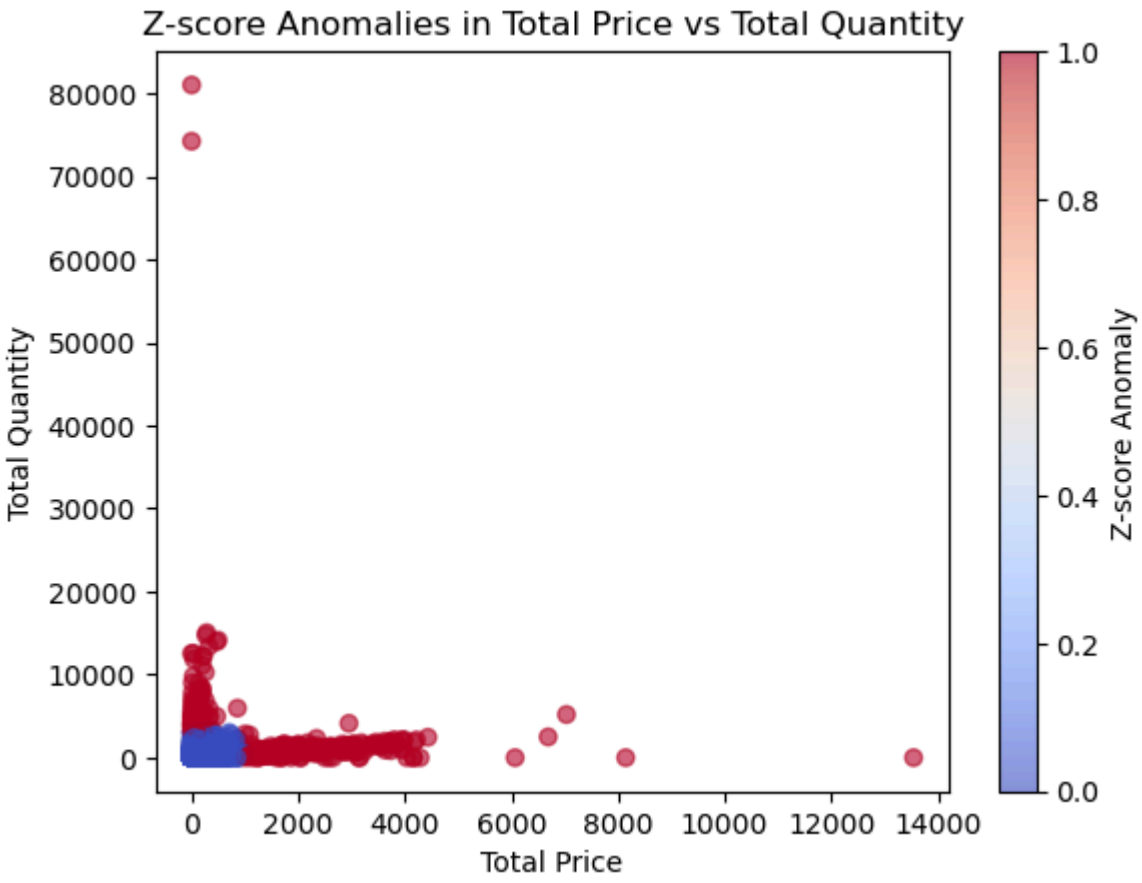
```
Price anomalies: 314
Quantity anomalies: 116
Total Z-score anomalies: 428

Example price anomalies:
           TotalPrice  TotalQuantity  Z_TotalPrice  Z_TotalQuantity  \
InvoiceNo
536544        2987.72           1208      9.720866         0.989925
536592        3612.35           1478     11.823119         1.275830
536865        1435.22            317      4.495778         0.046436
536876        3635.97           1641     11.902614         1.448432
537237        3713.99           1608     12.165198         1.413488

           ZScoreAnomaly  PriceAnomaly  QuantityAnomaly
InvoiceNo
536544                 1             1                0
536592                 1             1                0
536865                 1             1                0
536876                 1             1                0
537237                 1             1                0

Example quantity anomalies:
           TotalPrice  TotalQuantity  Z_TotalPrice  Z_TotalQuantity  \
InvoiceNo
536830           1.24           4280     -0.330419         4.242895
537659         118.87           3689      0.065476         3.617079
538991          72.53           3944     -0.090486         3.887101
539101          22.05           4800     -0.260381         4.793528
539338          37.37           3714     -0.208820         3.643552

           ZScoreAnomaly  PriceAnomaly  QuantityAnomaly
InvoiceNo
536830                 1             0                1
537659                 1             0                1
538991                 1             0                1
539101                 1             0                1
539338                 1             0                1
```

We can further show this by displaying it as a graph.

In this graph we color the anomalies red and the regular points blue.

In [44]:
```
# Visualize Z-score anomalies
plt.scatter(invoice_features["TotalPrice"], invoice_features["TotalQuantity"], c
plt.xlabel("Total Price")
plt.ylabel("Total Quantity")
plt.title("Z-score Anomalies in Total Price vs Total Quantity")
```

```
plt.colorbar(label="Z-score Anomaly")
plt.show()
```



7.2 - Isolation Forest

Isolation Forest is a multivariate anomaly detection method.
It constructs random decision trees and isolates observations that are **few and different** from the majority.

We apply it to the features `TotalPrice`, `AvgPrice`, `MaxPrice`, `StdPrice`, `TotalQuantity`, `NumUniqueProducts` and `AvgQuantityPerProduct`.

These features capture multiple aspecet of invoice-level behavior, including transaction value, price variation, basket size, and product diversity. By combining these features, Isolation Forest can detect invoices that are unusual **across multiple dimensions**, capturing subtle anomalies that a univariate method might miss.

We exclude `CustomerID`, `InvoiceDate` and `Country`, as these features do not directly describe transactional characteristics such as price or quantity. While they could proivide additional contextual insights, they are not central to identifying anomalous purchasing behavior in this analysis.

The Isolation Forest algorithm requires specifying the expected proportion of anomalies through the **contamination** parameter. In our case, it was set to 0.01, meaning that roughly 1% of observations/invoices are expected to be anomalous. This value was selected based on the assumption that anomalies in retail transaction data are relatively rare. Using a small contamination value ensures that the model focuses on identifying

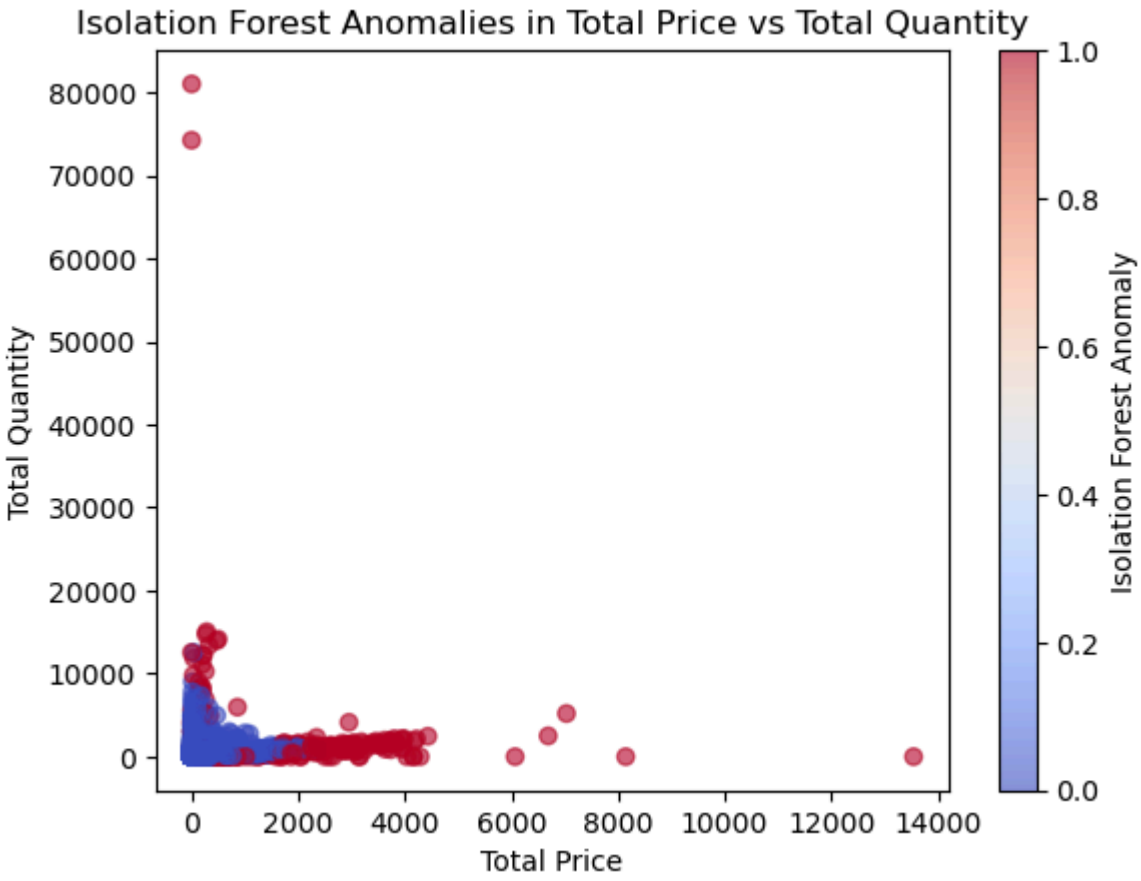
only the most unusual observations, reducing the risk of over-detecting normal transactions as anomalies.

In [45]:
```
# Define the potential features
features = [
    "TotalPrice", "AvgPrice", "MaxPrice", "StdPrice",
    "TotalQuantity", "NumUniqueProducts", "AvgQuantityPerProduct"
]

# Work on completed invoices only
filtered_invoices = invoice_features.loc[invoice_features["Cancelled"] == 0].cop

# Ensure model-ready values
filtered_invoices["StdPrice"] = filtered_invoices["StdPrice"].fillna(0.0)

# Scale features for better model stability
scaler = StandardScaler()
scaled_features = scaler.fit_transform(filtered_invoices[features])

# Fit isolation forest
iso_forest = IsolationForest(contamination=0.01, random_state=RANDOM_SEED)
iso_labels = iso_forest.fit_predict(scaled_features)

# Initialize and map anomalies back to main table
invoice_features["IsolationForestAnomaly"] = 0
invoice_features.loc[filtered_invoices.index, "IsolationForestAnomaly"] = (iso_l

# Print summary
print("Isolation Forest anomalies:", int(invoice_features.loc[filtered_invoices.

cols_to_show = [
    "TotalPrice", "TotalQuantity", "AvgPrice", "MaxPrice", "StdPrice",
    "NumUniqueProducts", "AvgQuantityPerProduct", "IsolationForestAnomaly"
]

print("\nExample isolation forest anomalies:")
print(invoice_features.loc[invoice_features["IsolationForestAnomaly"] == 1, cols
```

```
Isolation Forest anomalies: 208

Example isolation forest anomalies:
           TotalPrice  TotalQuantity  AvgPrice  MaxPrice   StdPrice  \
InvoiceNo
536544        2987.72           1208  5.669298    569.77  25.042269
536592        3612.35           1478  6.101943    607.49  25.256158
536830           1.24           4280  0.620000      1.06   0.622254
536876        3635.97           1641  6.131484    887.52  36.517475
537237        3713.99           1608  6.221089    863.74  35.533648

           NumUniqueProducts  AvgQuantityPerProduct  IsolationForestAnomaly
InvoiceNo
536544                    17               2.292220                       1
536592                    22               2.496622                       1
536830                     2            2140.000000                       1
536876                    23               2.767285                       1
537237                    24               2.693467                       1
```

We can further show this by displaying it as a graph.
In this graph we color the anomalies red and the regular points blue.

In [46]:
```
# Visualize isolation forest anomalies
plt.scatter(invoice_features["TotalPrice"], invoice_features["TotalQuantity"], c
plt.xlabel("Total Price")
plt.ylabel("Total Quantity")
plt.title("Isolation Forest Anomalies in Total Price vs Total Quantity")
plt.colorbar(label="Isolation Forest Anomaly")
plt.show()
```



7.3 - Comparison of Anomaly Detection Methods

To evaluate the robustness of the anomaly detection techniques, we compare the invoices flagged by the Z-score method and Isolation Forest.

The Z-score method detects extreme values in individual variables, while Isolation Forest identifies anomalies based on combinations of multiple features. As such, the two methods capture different types of anomalous behavior

By comparing the results, we can assess:

- the number of anomalies detected by each method
- the degree of overlap between the methods
- the characteristics of invoices uniquely identified by each approach

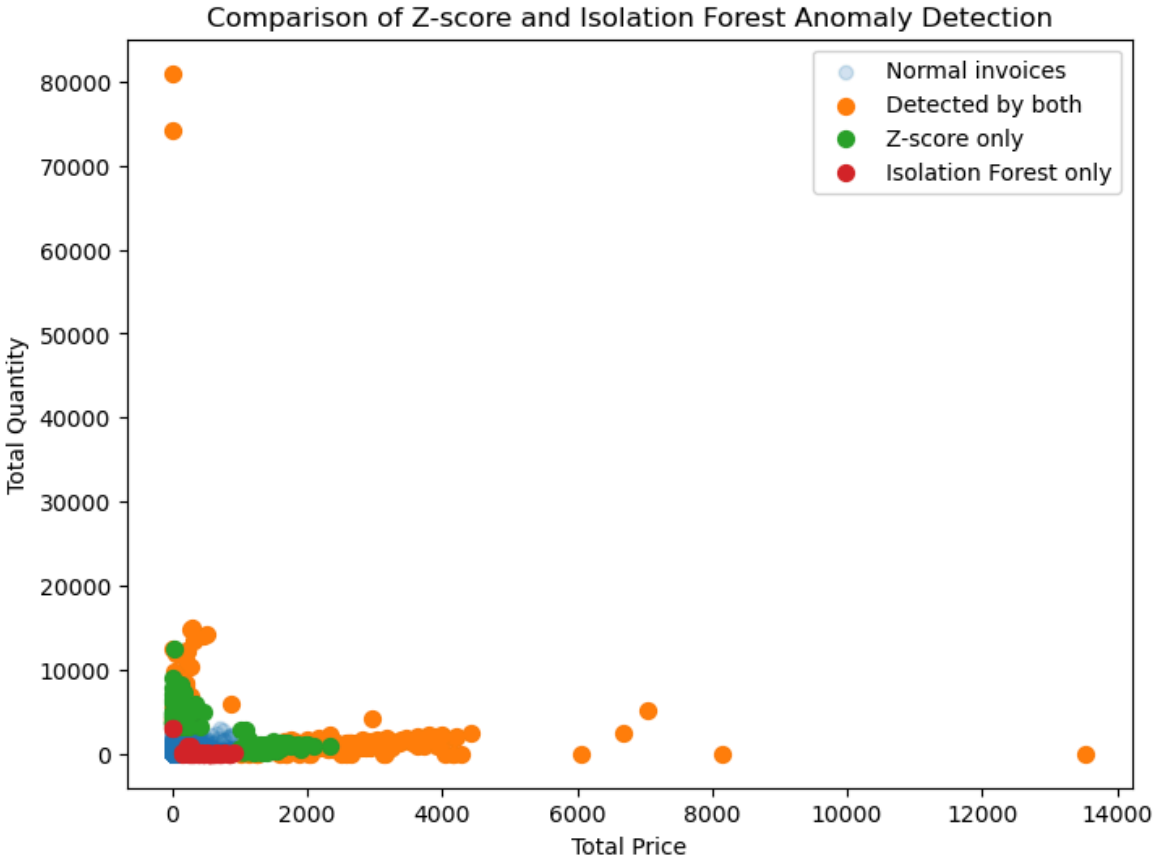
This comparison provides insight into whether the methods agree on the most extreme transactions, and whether they capture complementary aspects of anomalous behavior

In [47]:

```
# Use completed invoices for a clean apples-to-apples comparison
filtered_invoices = invoice_features.loc[invoice_features["Cancelled"] == 0].cop

# Counts
print("Z-score anomalies:", int(filtered_invoices["ZScoreAnomaly"].sum()))
print("Isolation Forest anomalies:", int(filtered_invoices["IsolationForestAnoma

# Crosstab comparison table
comparison = pd.crosstab(
    filtered_invoices["ZScoreAnomaly"],
    filtered_invoices["IsolationForestAnomaly"],
    rownames=["Z-score"],
    colnames=["Isolation Forest"]
)

print("\nComparison Table:")
print(comparison)

# Identify overlap groups
both_anomalies = filtered_invoices[
    (filtered_invoices["ZScoreAnomaly"] == 1) &
    (filtered_invoices["IsolationForestAnomaly"] == 1)
]
z_only = filtered_invoices[
    (filtered_invoices["ZScoreAnomaly"] == 1) &
    (filtered_invoices["IsolationForestAnomaly"] == 0)
]
iso_only = filtered_invoices[
    (filtered_invoices["ZScoreAnomaly"] == 0) &
    (filtered_invoices["IsolationForestAnomaly"] == 1)
]

print("\nOverlap Summary:")
print("Detected by both:", len(both_anomalies))
print("Only Z-score:", len(z_only))
print("Only Isolation Forest:", len(iso_only))

# Closer look at examples
print("\nExamples detected by BOTH methods:")
print(both_anomalies[["TotalPrice", "TotalQuantity"]].head())

print("\nExamples detected ONLY by Z-score:")
print(z_only[["TotalPrice", "TotalQuantity"]].head())

print("\nExamples detected ONLY by Isolation Forest:")
print(iso_only[["TotalPrice", "TotalQuantity"]].head())

# Visualization of anomaly overlaps
plt.figure(figsize=(8, 6))

# Normal points
plt.scatter(
    filtered_invoices["TotalPrice"],
    filtered_invoices["TotalQuantity"],
    alpha=0.2,
    label="Normal invoices"
 )

# Both methods
```

```
plt.scatter(
    both_anomalies["TotalPrice"],
    both_anomalies["TotalQuantity"],
    label="Detected by both",
    s=50
)

# Z-score only
plt.scatter(
    z_only["TotalPrice"],
    z_only["TotalQuantity"],
    label="Z-score only",
    s=50
)

# Isolation Forest only
plt.scatter(
    iso_only["TotalPrice"],
    iso_only["TotalQuantity"],
    label="Isolation Forest only",
    s=50
)

plt.xlabel("Total Price")
plt.ylabel("Total Quantity")
plt.title("Comparison of Z-score and Isolation Forest Anomaly Detection")
plt.legend()
plt.show()
```

```
Z-score anomalies: 428
Isolation Forest anomalies: 208

Comparison Table:
Isolation Forest      0    1
Z-score
0                 20256   41
1                   261  167

Overlap Summary:
Detected by both: 167
Only Z-score: 261
Only Isolation Forest: 41

Examples detected by BOTH methods:
           TotalPrice  TotalQuantity
InvoiceNo
536544        2987.72           1208
536592        3612.35           1478
536830           1.24           4280
536876        3635.97           1641
537237        3713.99           1608

Examples detected ONLY by Z-score:
           TotalPrice  TotalQuantity
InvoiceNo
536865        1435.22            317
537642        1384.25            256
538148        1406.05            225
538991          72.53           3944
539101          22.05           4800

Examples detected ONLY by Isolation Forest:
           TotalPrice  TotalQuantity
InvoiceNo
537534         924.59            115
539080         306.32            113
543253         298.75              5
543595         166.61            217
543688         145.85             50
```



7.4 Comparison Conclusion

The Z-score method detected 428 **anomaly invoices**, while Isolation Forest identified 208. Of these, 179 **invoices were detected by both methods**, indicating strong agreement on the most extreme transactions. This corresponds to roughly 86**%**, suggesting that the vast majority of anomalies detected by Isolation Forest are also identified by the Z-score method.

The Z-score method additionally flagged 249 **invoices** that were not identified by Isolation Forest. These transactions are likely extreme in individual variables, such as total price or quantity, but do not appear unusual when considering multiple features simultaneously.
This reflects the univariate nature of the Z-score methods, which is sensitive to large deviations in single dimensions.

In contrast, Isolation Forest detected only 29 **anomalies** that were not flagged by the Z-score method. These invoices were typically located closer to the dense region of the data distribution, suggesting that they represent more subtle anomalies appearing from unusual combinations of features, rather than extreme values in a single dimension.

Overall, the results highlight that the two methods are **complementary**. While the Z-score method is effective at identifying clearly extreme transactions, Isolation Forest is able to capture more complex and multivariate anomalies that would not be detected using a univariate approah.

This distinction is particulary relevant for retail transaction data, where anomalies may arise either from unusally large purchases or from atypical combinations of products and pricing, both of which are captured by the complementary methods used in this analysis.

7.5 Impact of Anomalies on Revenue

To further relate the anomaly detection results to the project objective, we analyze how much of the total revenue is generated by anomalous invoices. This allows us to determine whether a small number of extreme transactions contribute disproportionately to overall revenue.

In [48]:
```
# Total revenue
total_revenue = invoice_features["TotalPrice"].sum()

# Revenue from each method
z_revenue = invoice_features[
    invoice_features["ZScoreAnomaly"] == 1
]["TotalPrice"].sum()

iso_revenue = invoice_features[
    invoice_features["IsolationForestAnomaly"] == 1
]["TotalPrice"].sum()

# Combined anomalies (no double counting)
invoice_features["AnyAnomaly"] = (
    (invoice_features["ZScoreAnomaly"] == 1) |
    (invoice_features["IsolationForestAnomaly"] == 1)
).astype(int)

combined_revenue = invoice_features[
    invoice_features["AnyAnomaly"] == 1
]["TotalPrice"].sum()

# - Print results -
# Total revnue values
print(f"Total revenue: {total_revenue:.2f}")
print(f"Z-score anomaly revenue: {z_revenue:.2f}")
print(f"Isolation Forest anomaly revenue: {iso_revenue:.2f}")
print(f"Combined anomaly revenue: {combined_revenue:.2f}")

# Revenue share percentages
print("\nRevenue share:")
print(f"Z-score: {z_revenue / total_revenue:.2%}")
print(f"Isolation Forest: {iso_revenue / total_revenue:.2%}")
print(f"Combined: {combined_revenue / total_revenue:.2%}")

# Invoice counts and shares
print("\nInvoice counts:")
print(f"Total invoices: {len(invoice_features)}")
print(f"Anomalous invoices: {invoice_features['AnyAnomaly'].sum()}")
print(f"Share of invoices: {invoice_features['AnyAnomaly'].mean():.2%}")
```

```
Total revenue: 2060385.71
Z-score anomaly revenue: 629455.80
Isolation Forest anomaly revenue: 418488.86
Combined anomaly revenue: 648646.84

Revenue share:
Z-score: 30.55%
Isolation Forest: 20.31%
Combined: 31.48%

Invoice counts:
Total invoices: 20725
Anomalous invoices: 469
Share of invoices: 2.26%
```

Although anomalous invoices represent only a small fraction of all transactions, they account for approximately 31% **of total revenue**. This indicates that a limited number of extreme transactions contribute disproportionately to overall revenue.

8. Graph Based Analysis

8.1 Graph Representation

To complement the transaction-based clustering analysis, we reinterpret the dataset as a graph in order to capture relationships between products.

We construct a product co-occurrence graph, where:

Nodes represent products (StockCode)
Edges represent pairs of products purchased together within the same invoice
Edge weights represent the number of times two products co-occur accross all invoices

This representation allows us to model purchasing behavior as a network, where strong connections indicate products that are frequently bought together.

While co-occurence relationships can also be analyzed using traiditional pattern mining techniques, representing the data as a graph enables the application of graph mining algorithms, allowing us to uncover structural relationships between products

8.2 Graph Construction

We construct a graph by iterating over each invoice and connecting all pairs of products that appear together. This effectively builds a co-occurence network, where products frequently purchased together form stronger connections.

In [49]:
```
G = nx.Graph()

# Build co-occurrence edges
for invoice, group in data_clean.groupby("InvoiceNo"):
    products = group["StockCode"].unique()

    for p1, p2 in combinations(products, 2):
        if G.has_edge(p1, p2):
```

```
            G[p1][p2]["weight"] += 1
        else:
            G.add_edge(p1, p2, weight=1)
```

In [50]:
```
print("Nodes:", G.number_of_nodes())
print("Edges:", G.number_of_edges())
print(f"Density: {nx.density(G):.4f}")
print("Connected components:", nx.number_connected_components(G))
print(f"Avg degree: {sum(dict(G.degree()).values()) / G.number_of_nodes():.2f}")
lcc = max(nx.connected_components(G), key=len)
print(f"Largest component: {len(lcc)} nodes ({len(lcc)/G.number_of_nodes():.2%}
```

```
Nodes: 3914
Edges: 3751037
Density: 0.4898
Connected components: 1
Avg degree: 1916.73
Largest component: 3914 nodes (100.00% of total)
```

The unfiltered graph contains 3914 nodes (unique products) and $3,751,037$ edges, yielding an average degree of 1916. This extreme density is expected in a retail co-occurrence graph: popular products like bestselling gift items appear across thousands of invoices and therefore accumulate edges with nearly every other product in the catalogue.

The unfiltered graph exhibits a very high density of approximately 0.49, meaning that nearly half of all possible product pairs are connected by at least one co-occurrence relationship. This further confirms that the raw co-occurrence network is highly interconnected, largely due to popular products appearing across many invoices.

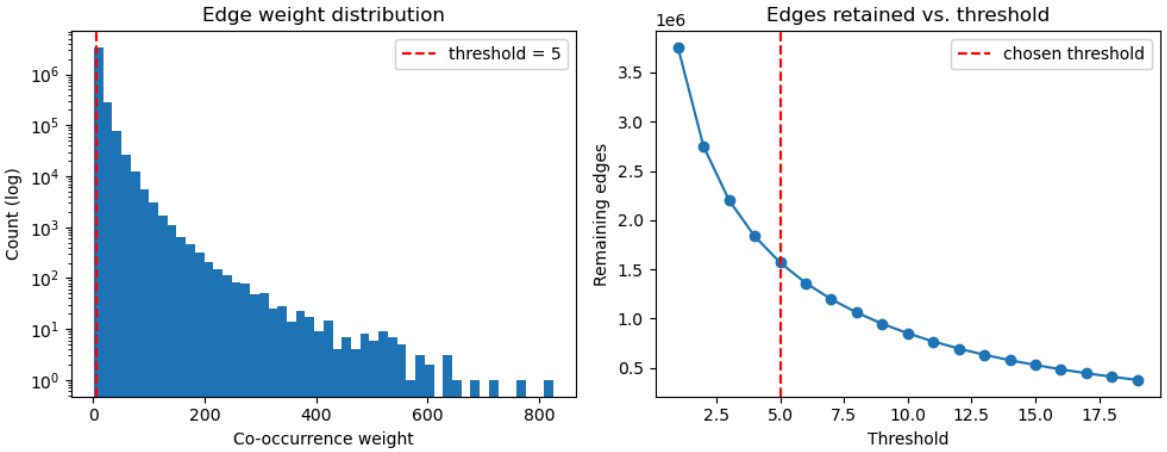
We can therefore conclude that the graph forms a giant single fully connected component — every product in the catalogue has been co-purchased with at least one other product at least once. This high level of connectivity further underscores why filtering is necessary: without a weight threshold, the graph offers no meaningful structural separation between products.

However, this construction leads to a very dense graph, as even infrequent co-occurrences introduce edges. In practice, this can obscure meaningful structure, as weak relationships create noise and connect otherwise unrelated products.

To reduce the influence of weak and potentially incidental co-occurrences, we apply a minimum edge-weight threshold. Only product pairs that co-occur at least a specified number of times are retained. This filtering process reduces graph density and helps emphasize stronger and more recurring purchasing relationships.

In [51]:
```
weights = [d["weight"] for u, v, d in G.edges(data=True)]

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.hist(weights, bins=50, log=True)
plt.axvline(5, color="red", linestyle="--", label="threshold = 5")
plt.xlabel("Co-occurrence weight")
plt.ylabel("Count (log)")
plt.title("Edge weight distribution")
```

```
plt.legend()

plt.subplot(1, 2, 2)
thresholds = range(1, 20)
edge_counts = [sum(1 for _, _, d in G.edges(data=True) if d["weight"] >= t) for
plt.plot(thresholds, edge_counts, "-o")
plt.axvline(5, color="red", linestyle="--", label="chosen threshold")
plt.xlabel("Threshold")
plt.ylabel("Remaining edges")
plt.title("Edges retained vs. threshold")
plt.legend()
plt.tight_layout()
plt.show()
```



In [52]:
```
# Filter weak edges (keep only meaningful co-occurrences)
threshold = 5

G_filtered = nx.Graph(
    (u, v, d) for u, v, d in G.edges(data=True)
    if d["weight"] >= threshold
)

print("Original edges:", G.number_of_edges())
print("Filtered edges:", G_filtered.number_of_edges())
```

```
Original edges: 3751037
Filtered edges: 1568711
```

After the filter we get:

In [53]:
```
print("Filtered nodes:", G_filtered.number_of_nodes())
print("Filtered edges:", G_filtered.number_of_edges())
print(f"Filtered density: {nx.density(G_filtered):.4f}")
print(f"Filtered avg degree: {sum(dict(G_filtered.degree()).values()) / G_filter
```

```
Filtered nodes: 3263
Filtered edges: 1568711
Filtered density: 0.2948
Filtered avg degree: 961.51
```

In [54]:
```
print(f"Global clustering coefficient: {nx.transitivity(G_filtered):.4f}")
```

```
Global clustering coefficient: 0.6836
```

Due to the large size and density of the graph, we compute the global clustering coefficient, instead of the average clustering coeffecient, which provides a scalable

measure of local interconnectedness.

The filtered graph exhibits a high global clustering coefficient of approximately 0.68, indicating that products connected to the same product are themselves frequently connected. This suggests the presence of tightly interconnected product groups and reflects the tendency of retail purchases to form overlapping basket structures.

At the same time, the graph remains relatively dense even after filtering, with a density of approximately 0.29, meaning that many of these local product groups overlap substantially. This helps explain the relatively low modularity values observed during community detection: while meaningful local structure exists, the boundaries between communities remain weak due to the highly interconnected nature of the retail network.

The weight distribution confirms that the majority of co-occurrences are incidental (weight $1 - 4$), and the retention curve shows a clear drop-off before threshold 5, supporting our choice.

To further validate the choice of a threshold value of 5, we perform a threshold sensitivity analysis across multiple edge-weight values.

8.2.1 - Threshold Sensitivity Analysis

To evaluate the effect of different edge-weight thresholds, we perform a sensitivity analysis across multiple threshold values.
For each threshold, we reconstruct the filtered graph and apply Louvain community detection, recording the resulting graph size, number of detected communities, and modularity.

In [55]:
```
threshold_results = []

threshold_values = [1, 3, 5, 7, 10]

for t in threshold_values:

    # Build filtered graph
    G_t = nx.Graph(
        (u, v, d)
        for u, v, d in G.edges(data=True)
        if d["weight"] >= t
    )

    # Skip empty graphs
    if G_t.number_of_nodes() == 0:
        continue

    # Louvain partition
    partition_t = community_louvain.best_partition(
        G_t,
        weight="weight",
        random_state=RANDOM_SEED
    )

    # Community count
```

```
    num_communities = len(set(partition_t.values()))

    # Modularity
    modularity_t = community_louvain.modularity(
        partition_t,
        G_t,
        weight="weight"
    )

    # Avg degree
    avg_degree = (
        sum(dict(G_t.degree()).values())
        / G_t.number_of_nodes()
    )

    threshold_results.append({
        "threshold": t,
        "nodes": G_t.number_of_nodes(),
        "edges": G_t.number_of_edges(),
        "avg_degree": avg_degree,
        "communities": num_communities,
        "modularity": modularity_t
    })

threshold_df = pd.DataFrame(threshold_results)

threshold_df
```

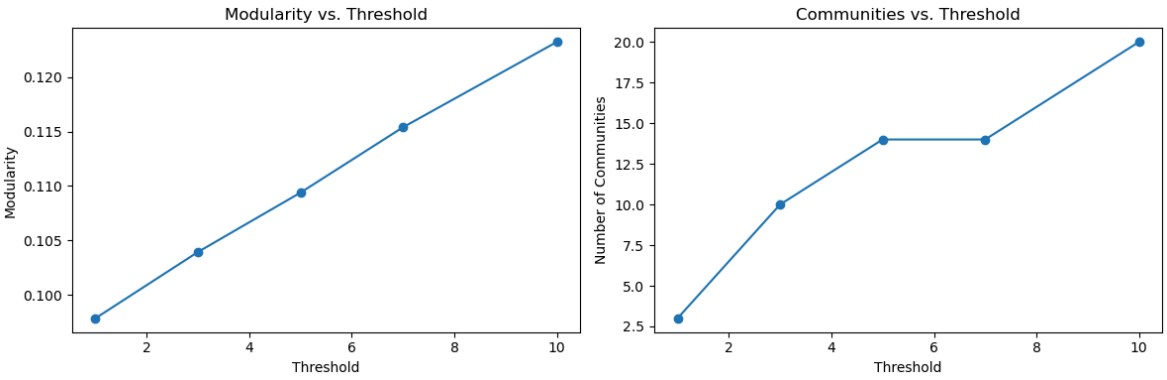
Out[55]:

	threshold	nodes	edges	avg_degree	communities	modularity
0	1	3914	3751037	1916.728155	3	0.097838
1	3	3538	2201082	1244.252120	10	0.103936
2	5	3263	1568711	961.514557	14	0.109390
3	7	3087	1196302	775.057985	14	0.115401
4	10	2847	850028	597.139445	20	0.123241

In [56]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Modularity
axes[0].plot(
    threshold_df["threshold"],
    threshold_df["modularity"],
    marker="o"
)
axes[0].set_xlabel("Threshold")
axes[0].set_ylabel("Modularity")
axes[0].set_title("Modularity vs. Threshold")

# Communities
axes[1].plot(
    threshold_df["threshold"],
    threshold_df["communities"],
    marker="o"
)
axes[1].set_xlabel("Threshold")
axes[1].set_ylabel("Number of Communities")
```

```
axes[1].set_title("Communities vs. Threshold")

plt.tight_layout()
plt.show()
```



Based on this analysis we can see that as the threshold increases, weak co-occurrence relationships are progressively removed, substantially reducing graph density and average node degree. At the same time, modularity gradually increases, indicating slightly stronger separation between communities.

However, the increase in modularity remains relatively small even after removing a large number of edges. For example, increasing the threshold from 1 to 10 reduces the number of edges from approximately 3.75 million to 0.85 million, while modularity increases only from 0.098 to 0.123. This suggests that the graph remains inherently highly interconnected even after substantial filtering, reflecting the overlapping nature of retail purchasing behavior.

The number of detected communities also increases with higher thresholds, as weaker inter-community connections are removed and product groups become more distinct. Notably, the community structure remains relatively stable between thresholds 5 and 7, both yielding 14 communities.

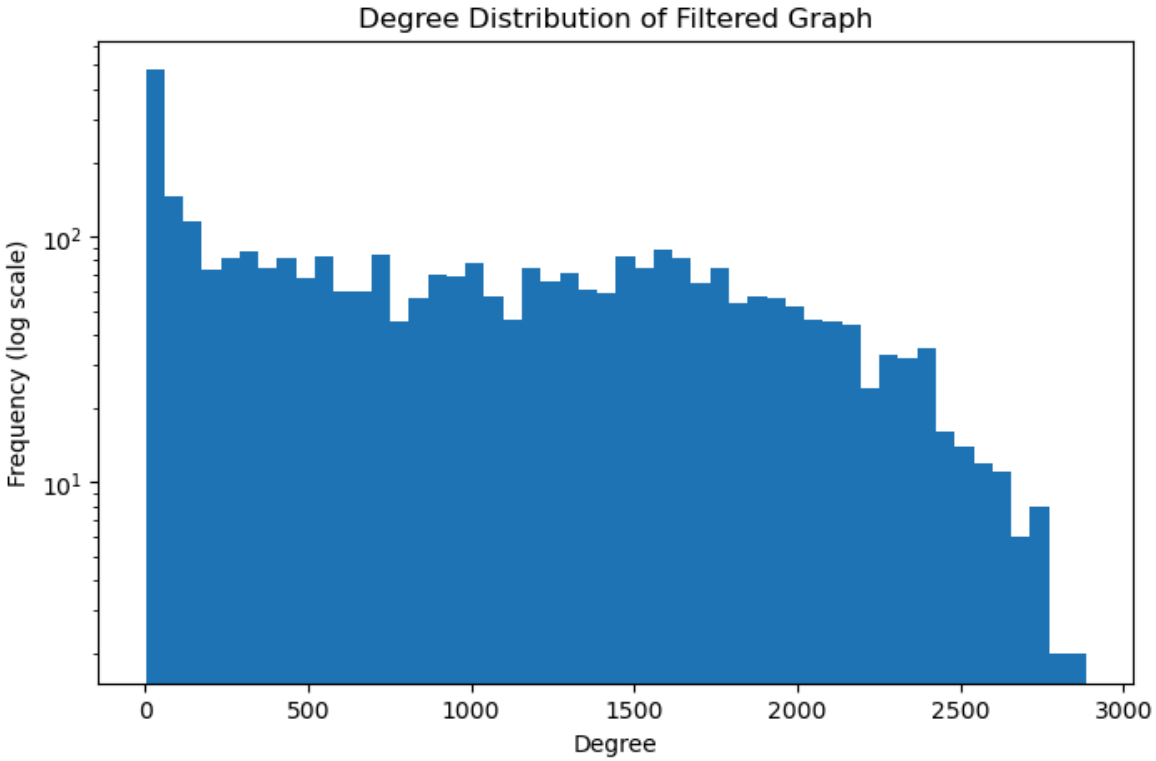
Based on this analysis, a threshold value of 5 provides a reasonable balance between reducing incidental co-occurrences and preserving sufficient graph structure for meaningful community analysis.

8.2.2 Degree Distribution

To further characterize the structure of the filtered graph, we examine the distribution of node degrees.
The degree of a product corresponds to the number of other products with which it frequently co-occurs.

In [57]:
```
degrees = [d for _, d in G_filtered.degree()]

plt.figure(figsize=(8, 5))

plt.hist(degrees, bins=50, log=True)

plt.xlabel("Degree")
plt.ylabel("Frequency (log scale)")
```

```
plt.title("Degree Distribution of Filtered Graph")

plt.show()
```



The degree distribution of the filtered graph is highly skewed, with a small number of products connected to a very large portion of the network. Even after thresholding, several products maintain extremely high degrees, indicating that certain popular retail items frequently co-occur with many other products.

This hub-like structure contributes to the overall density of the graph and increases overlap between product groups. It also helps explain the relatively low modularity score observed in Section **8.3.1**.

8.3 Community Detection

To identify different groups of products that are frequently being purchased together, we apply **Community Detection** using the **Louvain method** and **Clique Percolation**.

Community Detection aims to partition the graph into groups of nodes that are more densely connected internally than with the rest of the graph. In this context, a community corresponds to a set of products that are often purchased together.

8.3.1 Louvain Method

The Louvain algorithm was selected due to its efficiency and its ability to detect meaningful communities in large, weighted networks. It works by maximizing the modularity, which measures how well the graph is partitioned into densely connected subgroups.

In [58]:
```
partition = community_louvain.best_partition(G_filtered, weight="weight", random
```

In [59]:
```
modularity = community_louvain.modularity(partition, G_filtered, weight="weight"
print(f"Modularity: {modularity:.4f}")
```

```
Modularity: 0.1094
```

The modularity score of 0.1094 is lower than the "in practice" threshold of $0.3 - 0.7$ for well-separated communities. However, this is expected given the structural properties of the filtered graph: even after applying the weight threshold, the graph retains a high average degree of ~962, meaning most products still share edges with a large fraction of all other nodes.

In such dense networks, the random graph baseline used by modularity is itself dense, compressing the range of achievable scores significantly. The low score therefore reflects the density of the data rather than a failure of community detection — the Louvain algorithm is still partitioning the graph into the most internally cohesive groups achievable given these constraints.

8.4 Organizing the Communities

The resulting partition assigns each product to a community. We then group products by their assignment community for better interpretation.

In [60]:
```
communities = defaultdict(list)

for product, comm_id in partition.items():
    communities[comm_id].append(product)

print("Number of communities:", len(communities))
```

```
Number of communities: 14
```

8.5 Inspecting the Communities

To better understand the structure of the graph, we inspect the largest communities. Focusing on the largest communities allows us to identify the dominant product groupings, while smaller commnities typically correspond to more niche or infrequently co-purchased items.

In [61]:
```
sorted_communities = sorted(communities.items(), key=lambda x: len(x[1]), revers

for comm_id, products in sorted_communities[:5]:    # Currently shows top 5
    print(f"\nCommunity {comm_id} (size {len(products)}):")
    print(products[:10])
```

```
Community 1 (size 1306):
['85123A', '71053', '84406B', '84029G', '84029E', '22752', '20679', '37370', '218
71', '21071']

Community 2 (size 1012):
['21730', '22941', '22963', '22910', '20668', '22654', '84030E', '22633', '2286
6', '22865']

Community 3 (size 920):
['22962', '84970S', '22198', '21506', '21328', '22473', '84509A', '84510A', '2211
7', '20966']

Community 10 (size 4):
['37489D', '37489B', '37489C', '37489A']

Community 0 (size 3):
['23305', '23303', '23304']
```

While the StockCode provides a unique identifier for each product, it is not directly interpretable. To improve readability and better understand the nature of each of the communities, we map each StockCode to it's corresponding product description.

In [62]:
```python
# Create mapping from StockCode to Description
product_map = data_clean.drop_duplicates("StockCode").set_index("StockCode")["De

# Print communities with product names
for comm_id, products in sorted_communities[:5]:
    print(f"\nCommunity {comm_id} (size {len(products)}):")

    for p in products[:10]:
        print(product_map.get(p, p))
```

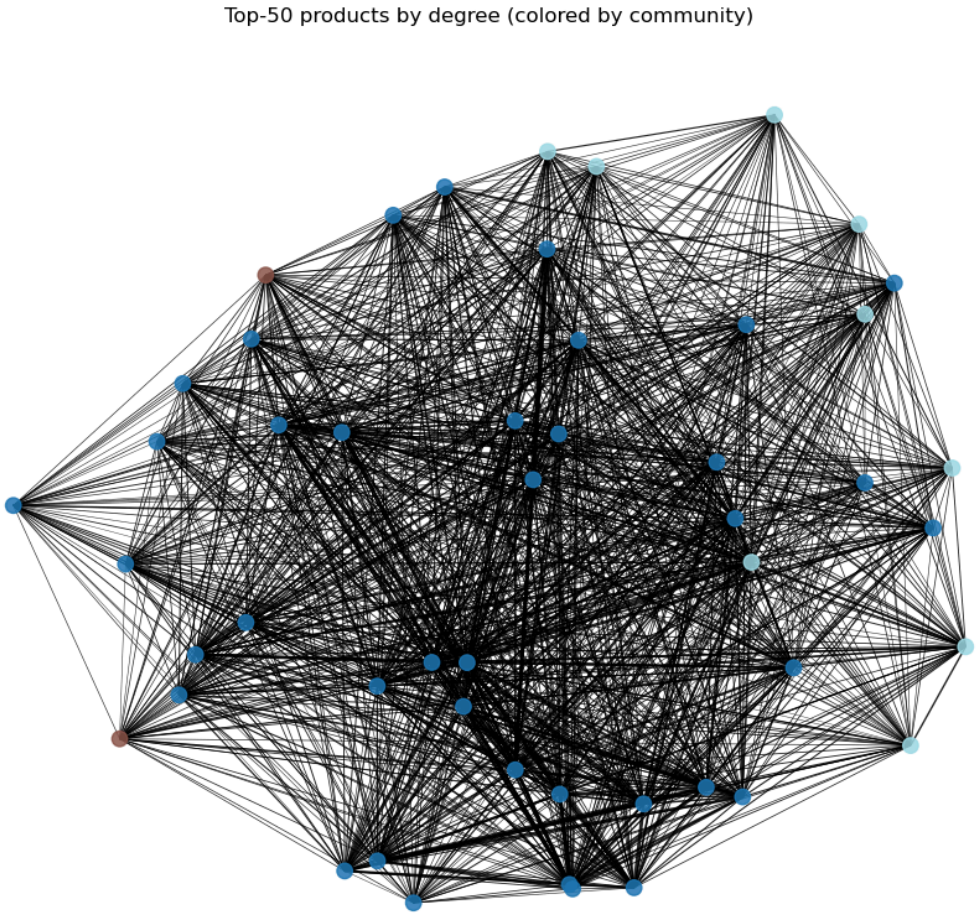
```
Community 1 (size 1306):
WHITE HANGING HEART T-LIGHT HOLDER
WHITE METAL LANTERN
CREAM CUPID HEARTS COAT HANGER
KNITTED UNION FLAG HOT WATER BOTTLE
RED WOOLLY HOTTIE WHITE HEART.
SET 7 BABUSHKA NESTING BOXES
EDWARDIAN PARASOL RED
RETRO COFFEE MUGS ASSORTED
SAVE THE PLANET MUG
VINTAGE BILLBOARD DRINK ME MUG

Community 2 (size 1012):
GLASS STAR FROSTED T-LIGHT HOLDER
CHRISTMAS LIGHTS 10 REINDEER
JAM JAR WITH GREEN LID
PAPER CHAIN KIT VINTAGE CHRISTMAS
DISCO BALL CHRISTMAS DECORATION
DELUXE SEWING KIT
ENGLISH ROSE HOT WATER BOTTLE
HAND WARMER UNION JACK
HAND WARMER SCOTTY DOG DESIGN
HAND WARMER OWL DESIGN

Community 3 (size 920):
JAM JAR WITH PINK LID
HANGING HEART ZINC T-LIGHT HOLDER
LARGE POPCORN HOLDER
FANCY FONT BIRTHDAY CARD,
BALLOONS  WRITING SET
TV DINNER TRAY VINTAGE PAISLEY
SET OF 4 ENGLISH ROSE PLACEMATS
SET OF 4 ENGLISH ROSE COASTERS
METAL SIGN HER DINNER IS SERVED
SANDWICH BATH SPONGE

Community 10 (size 4):
PINK/GREEN FLOWER DESIGN BIG MUG
BLUE/YELLOW FLOWER DESIGN BIG MUG
GREEN/BLUE FLOWER DESIGN BIG MUG
YELLOW/PINK FLOWER DESIGN BIG MUG

Community 0 (size 3):
SET 4 PICNIC CUTLERY BLUEBERRY
SET 4 PICNIC CUTLERY FONDANT
SET 4 PICNIC CUTLERY CHERRY
```

This allows us to better understand the composition of the communities, and identify common themes, such as groups of related items or product variants. So, by examining the product descriptions, we can better interpret the underlying purchasing patterns captured by the graph.

8.6 Graph Visualization

In [63]:
```python
# Visualize subgraph of top 50 nodes by degree
top_nodes = sorted(G_filtered.degree(), key=lambda x: x[1], reverse=True)[:50]
top_node_ids = [n for n, _ in top_nodes]
subgraph = G_filtered.subgraph(top_node_ids)
```

```
node_colors = [partition.get(n, 0) for n in subgraph.nodes()]
edge_weights = [subgraph[u][v]["weight"] for u, v in subgraph.edges()]

plt.figure(figsize=(12, 10))
pos = nx.spring_layout(subgraph, seed=RANDOM_SEED)
nx.draw_networkx(
    subgraph, pos,
    node_color=node_colors, cmap="tab20",
    width=[w / max(edge_weights) * 3 for w in edge_weights],
    with_labels=False,
    node_size=80, alpha=0.85
)
plt.title("Top-50 products by degree (colored by community)")
plt.axis("off")
plt.show()
```



To visualize the graph, we extract the 50 highest-degree products, as these are the most central nodes and most representative of the overall structure.

A spring layout is used, which positions strongly connected nodes closer together, making community structure visually apparent. Nodes are colored by their community assignment from the Louvain partition, and edge thickness is scaled proportionally to co-occurrence weight. The full graph is too dense to visualize meaningfully, so this subgraph serves as a representative window into the dominant structure.

8.7 Interpretation

The detected communities represent groups of products that are frequently purchased together. These communities can be interpreted as product groupings or themse, reflecting underlying purchasing patterns in the dataset.

The resulting structure consists of a few large communities alongside several smaller ones.
The large communities represent dominant product groupings, where a wide range of products are frequently co-purchased. These likely correspond to broad shopping patterns, such as general gift items, home decorations, or other commonly bundled retail products.

In contrast, the smaller communities typically consist of only a few products with very similar StockCodes. These are often closely related products variants - forexample same item with different version or variant - which are frequently purchased together, but have limited interaction with other products. This shows that the graph captures both broad purchasing behavior, but also very specific product level relationships.

Products within the same community tend to co-occur frequently in invoices, while products from different communities are less likely to be purchased together. This suggests that the community detection is able to identify groups of strongly related products based on co-occurence patterns.

Community detection provides a complementary perspective to the clustering approach, which groups entire transactions based on aggregated features. While clustering captures transaction level behavior, the graph-based approach captures relationships between individual products, offering a more precise view of purchasing patterns.

Overall, the results suggest that purchasing behavior is structured around a limited number of dominant product groupings, alongside smaller, more specialized product relationships. This highlights the presence of both broad and niche purchasing patterns within the dataset, which are effectively captured through the graph-based approach.

8.8 Clique Percolation Method

To further explore the structure of the product network, we also apply clique percolation community detection. Unlike the Louvain method, which partitions the graph into disjoint communities, clique percolation identifies overlapping communities by searching for densely connected groups of products. This allows products to belong to multiple communities simultaneously, which is relevant in retail settings where a product may be associated with several purchasing patterns.

The method identifies communities by locating fully connected subgraphs (cliques) of size k and merging them when they overlap. The choice of k therefore strongly affects the resulting community structure. Smaller values, such as $k = 3$ or $k = 4$, are generally too permissive in dense graphs, as the abundance of small cliques can cause large portions of the network to merge into one broad overlapping community.

Given the high density and clustering coefficient observed in the previous graph diagnostics, we select $k = 5$. This provides a stricter definition of community membership by requiring stronger local connectivity, reducing incidental overlap caused by highly connected hub products. The chosen value therefore balances structural strictness with interpretability.

Because clique percolation is computationally expensive on dense graphs, we apply the method to a representative subgraph rather than the full filtered network. Specifically, we select the largest Louvain community and extract the 300 highest-degree products within that group. This preserves the densest local product structure while keeping the computation tractable.

In [64]:
```
# Select largest Louvain community
largest_comm_id = max(
    communities,
    key=lambda c: len(communities[c])
)

largest_nodes = communities[largest_comm_id]

G_sub = G_filtered.subgraph(largest_nodes).copy()

# Reduce to top-N nodes
N = 300
k = 5

top_nodes = sorted(
    G_sub.degree(),
    key=lambda x: x[1],
    reverse=True
)[:N]

selected_nodes = [n for n, _ in top_nodes]

G_sub_test = G_sub.subgraph(selected_nodes).copy()

print("Nodes:", G_sub_test.number_of_nodes())
print("Edges:", G_sub_test.number_of_edges())

clique_communities = list(
    k_clique_communities(G_sub_test, k)
)

print("Number of communities:", len(clique_communities))

sizes = sorted(
    [len(c) for c in clique_communities],
    reverse=True
)

print("Largest sizes:", sizes[:10])
```

```
Nodes: 300
Edges: 44826
Number of communities: 1
Largest sizes: [300]
```

The reduced subgraph contains 300 nodes and $44,826$ edges. Applying clique percolation with $k = 5$ identified a single overlapping community containing all 300 products.

This indicates that the selected products form a highly interconnected local structure, where many 5-cliques overlap sufficiently to merge into one large clique-based community. In practice, this means that the densest region of the product graph does not contain clearly separated overlapping communities, but instead forms one strongly connected product cluster.

This result complements the Louvain analysis. While Louvain partitions the graph into separate communities, clique percolation shows that within the largest product group, substantial overlap remains between products. The two methods therefore capture different aspects of the same structure: Louvain identifies broader partitions, while clique percolation reveals that dense local regions contain many overlapping relationships.

Overall, the clique percolation result reinforces the earlier interpretation that the retail co-occurrence network is characterized by strong local connectivity and substantial overlap between product groups. This also helps explain the relatively low modularity score observed for Louvain, as highly connected products create bridges between otherwise distinct communities.

8.9 Relationship Between Communities and Anomalies

To further connect the graph-based analysis with the anomaly detection results, we examine how anomalous invoices are distributed across product communities.

For each invoice, we compute the number of distinct Louvain communities represented among its purchased products. This provides a measure of the structural diversity of an invoice within the product co-occurrence network.

Invoices containing products from many different communities may reflect unusually broad or mixed purchasing behavior, while invoices concentrated within fewer communities may correspond to more specialized purchases.

In [65]:
```
invoice_community_counts = []

for invoice, group in data_clean.groupby("InvoiceNo"):

    # Products in invoice
    products = group["StockCode"].unique()

    # Communities represented
    comms = {
        partition[p]
        for p in products
        if p in partition
    }

    invoice_community_counts.append({
        "InvoiceNo": invoice,
```

```
        "NumCommunities": len(comms)
    })

community_spread_df = pd.DataFrame(
    invoice_community_counts
)
```

We then combine the community diversity information with the anomaly detection results obtained previously. This allows us to compare whether anomalous invoices differ structurally from normal invoices in terms of the variety of product communities represented.

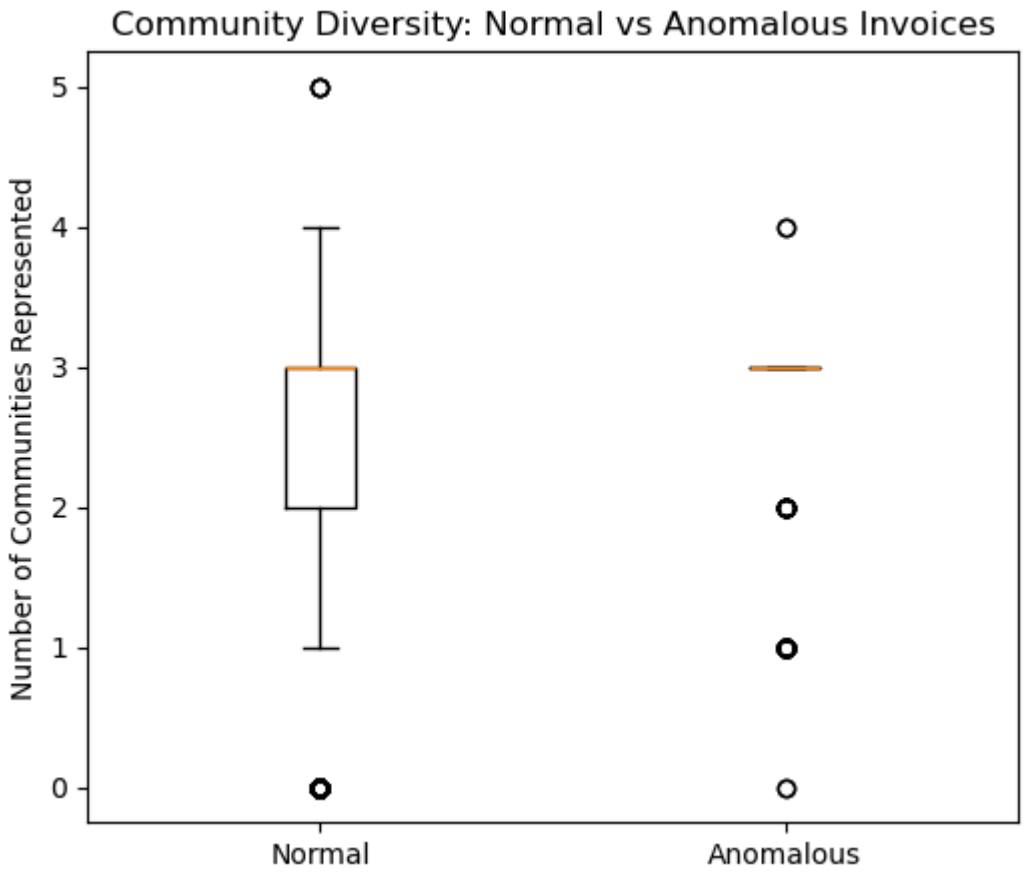
In [66]:
```
# Create combined anomaly label if not already present
invoice_features["AnyAnomaly"] = (
    (invoice_features["ZScoreAnomaly"] == 1) |
    (invoice_features["IsolationForestAnomaly"] == 1)
).astype(int)

community_spread_df = community_spread_df.merge(
    invoice_features[["AnyAnomaly"]],
    left_on="InvoiceNo",
    right_index=True,
    how="left"
)
```

In [67]:
```
normal_spread = community_spread_df[
    community_spread_df['AnyAnomaly'] == 0
]["NumCommunities"]

anomaly_spread = community_spread_df[
    community_spread_df['AnyAnomaly'] == 1
]["NumCommunities"]

print("Normal invoices:")
print(normal_spread.describe())

print("\nAnomalous invoices:")
print(anomaly_spread.describe())
```

```
Normal invoices:
count    20256.000000
mean         2.282929
std          0.827613
min          0.000000
25%          2.000000
50%          3.000000
75%          3.000000
max          5.000000
Name: NumCommunities, dtype: float64

Anomalous invoices:
count    469.000000
mean       2.633262
std        0.755211
min        0.000000
25%        3.000000
50%        3.000000
75%        3.000000
max        4.000000
Name: NumCommunities, dtype: float64
```

In [68]:
```python
plt.figure(figsize=(6, 5))
plt.boxplot(
    [normal_spread, anomaly_spread],
    labels=["Normal", "Anomalous"]
)
plt.ylabel("Number of Communities Represented")
plt.title("Community Diversity: Normal vs Anomalous Invoices")
plt.show()

print("Normal invoices – median:", normal_spread.median())
print("Anomalous invoices – median:", anomaly_spread.median())
```

```
C:\Users\hjalt\AppData\Local\Temp\ipykernel_18624\2724894187.py:2: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot(
```



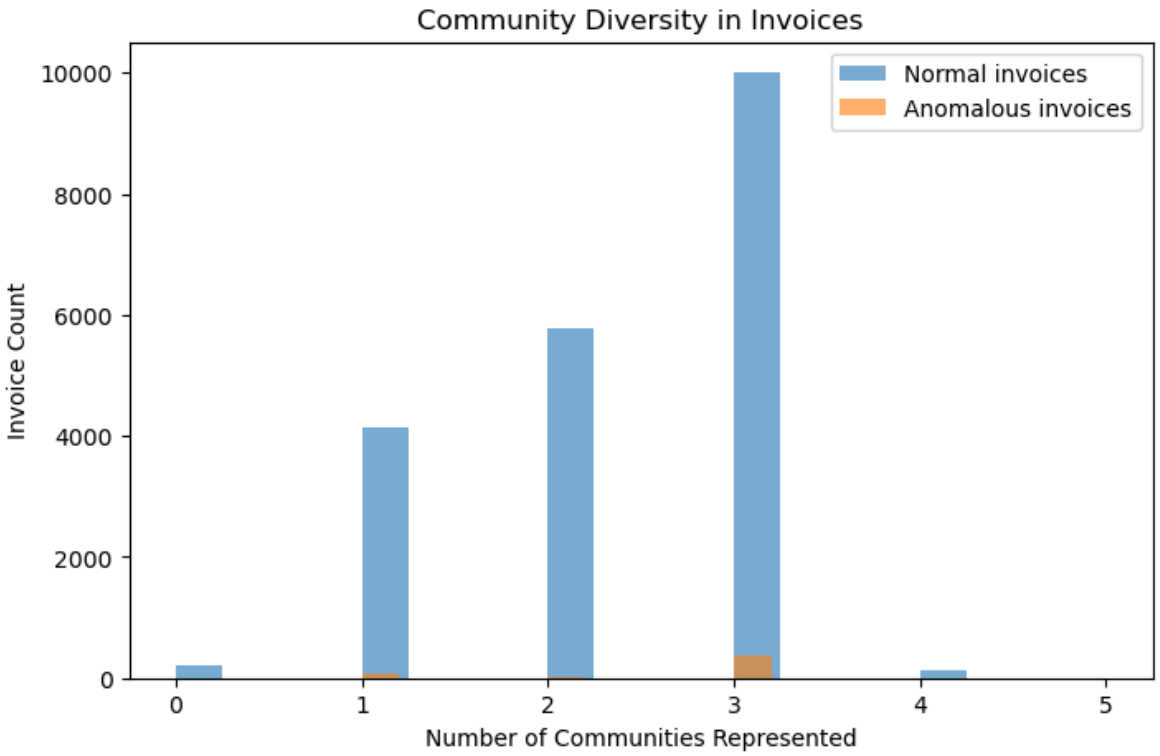

```
Normal invoices – median: 3.0
Anomalous invoices – median: 3.0
```

Although both groups share the same median number of represented communities (3), anomalous invoices are more tightly concentrated around the median, indicating lower variability in their structural composition. In contrast, normal invoices exhibit a broader spread in the number of represented communities. The medians and descriptive statistics printed above confirm this pattern numerically.

To further compare the structural composition of invoices, we visualize the distribution of the number of represented communities for normal and anomalous invoices.

In [69]:
```
plt.figure(figsize=(8,5))

plt.hist(
    normal_spread,
    bins=20,
    alpha=0.6,
    label="Normal invoices"
)

plt.hist(
    anomaly_spread,
    bins=20,
    alpha=0.6,
    label="Anomalous invoices"
)

plt.xlabel("Number of Communities Represented")
plt.ylabel("Invoice Count")
plt.title("Community Diversity in Invoices")
```

```
plt.legend()
plt.show()
```



The histogram further supports this observation, showing that anomalous invoices tend to have a higher count of represented communities, consistent with the median difference seen in the box plot above.

This finding complements the earlier anomaly detection analysis by suggesting that unusual invoices are not only extreme in terms of price or quantity, but also structurally unusual within the product network. In other words, anomalous transactions tend to connect multiple product communities simultaneously, indicating more complex purchasing behavior than typical invoices.

Overall, this analysis suggests that anomalous transactions are not only statistically unusual, but also structurally distinct within the product co-occurrence network, reinforcing the value of combining graph mining with anomaly detection techniques.

8.10 Comparison with the Clustering

The clustering analysis and graph-based analysis provide complementary perspectives on purchasing behavior.

The clustering approach focused on transaction level patterns by grouping invoices according to aggregated behavioral features such as purchase quantity and transaction value. This revealed broader customer purchasing trends, including the dominance of high-volume UK transactions.

In contrast, the graph-based approach focused on relationships between individual products. By modeling co-occurrence patterns directly, the graph analysis identified

groups of products that were frequently purchased together and revealed structural relationships within the product catalogue.

While clustering captures similarities between transactions, graph mining captures associations between products. Together, these approaches provide a more complete understanding of customer purchasing behavior at both the transaction and product level.

This analysis may help the company identify which customer groups and product categories should be prioritized in future sales and marketing efforts.

9. Pattern Mining

The purpose of this section is to identify recurring purchasing patterns within customer transactions using frequent itemset mining and association rule learning.

While the clustering analysis focused on grouping invoices according to aggregated behavioral features, pattern mining instead focuses on discovering direct relationships between products that frequently appear together in the same transaction.

To achieve this, we apply the Apriori algorithm to identify frequent itemsets and subsequently generate association rules describing product co-occurrence relationships.

These patterns may provide insight into:

- complementary products,
- recurring purchasing behavior,
- potential recommendation opportunities,
- and products groupings useful for marketing or store organization.

9.1 Transaction Construction

To perform pattern mining, the dataset is transformed into a transaction-based representation.

Each invoice is treated as a single transaction basket containing the products purchased together in that invoice.

Unlike the clustering section, where invoices were represented using aggregated numerical features, the pattern mining approach only requires the set of purchased products for each transaction. Therefore, the dataset is grouped by InvoiceNo, and each transaction is represented as a list of purchased StockCode values.

We use `StockCode` instead of product descriptions because it provides a cleaner and more consistent product identifier.

In [70]:
```
transactions = data_clean.groupby("InvoiceNo")["StockCode"].apply(list).tolist()
```

```
print(f"Number of transactions: {len(transactions)}")
transactions[:3]
```

```
Number of transactions: 20725
```

Out[70]:
```
[['85123A', '71053', '84406B', '84029G', '84029E', '22752', '21730'],
 ['22633', '22632'],
 ['84879',
  '22745',
  '22748',
  '22749',
  '22310',
  '84969',
  '22623',
  '22622',
  '21754',
  '21755',
  '21777',
  '48187']]
```

9.2 Transaction Encoding

The Apriori algorithm requires transactional data in binary format.

Using `TransactionEncoder`, the transactions are transformed into a sparse binary matrix where:

- rows represent invoices,
- columns represent products,
- and values indicate whether a product appears in a transaction.

We use this later on in apriori algorithm.

In [71]:
```
encoder = TransactionEncoder()
encoded_array = encoder.fit(transactions).transform(transactions)
data_encoded = pd.DataFrame(encoded_array,columns=encoder.columns_)
data_encoded.head()
```

Out[71]:

	10002	10080	10120	10123C	10124A	10124G	10125	10133	10135	11001	...
0	False	False	False	False	False	False	False	False	False	False	...
1	False	False	False	False	False	False	False	False	False	False	...
2	False	False	False	False	False	False	False	False	False	False	...
3	False	False	False	False	False	False	False	False	False	False	...
4	False	False	False	False	False	False	False	False	False	False	...

5 rows × 3940 columns

9.3 Frequent Itemset Mining using Apriori

The Apriori algorithm is used to identify combinations of products that occur frequently across transactions.

A minimum support threshold of 2% was selected after experimentation. We found that higher support thresholds produced very few frequent itemsets due to the sparse and highly diverse nature of the retail dataset.
So if we would have used a higher support threshold, it would have become impossible to create association rules.

This is expected because:

- the dataset contains many unique products,
- customer purchasing behavior is diverse,
- and most products only appear in a small subset of transactions.

The low percentage of minimum support is explained in the graph mining section, where we discovered that our graph has low modularity.

In [72]:
```
pattern = apriori(data_encoded, min_support=0.02, use_colnames=True)

pattern = pattern.sort_values(
    by='support',
    ascending=False
)

print("Total Frequent Itemsets:", pattern.shape[0])
pattern
```

```
Total Frequent Itemsets: 353
```

Out[72]:

	support	itemsets
271	0.106297	frozenset({85123A})
268	0.100941	frozenset({85099B})
112	0.095971	frozenset({22423})
230	0.081351	frozenset({47566})
11	0.075513	frozenset({20725})
...	...	...
150	0.020024	frozenset({22661})
228	0.020024	frozenset({23493})
282	0.020024	frozenset({20723, 22355})
303	0.020024	frozenset({23206, 20727})
318	0.020024	frozenset({85099B, 22379})

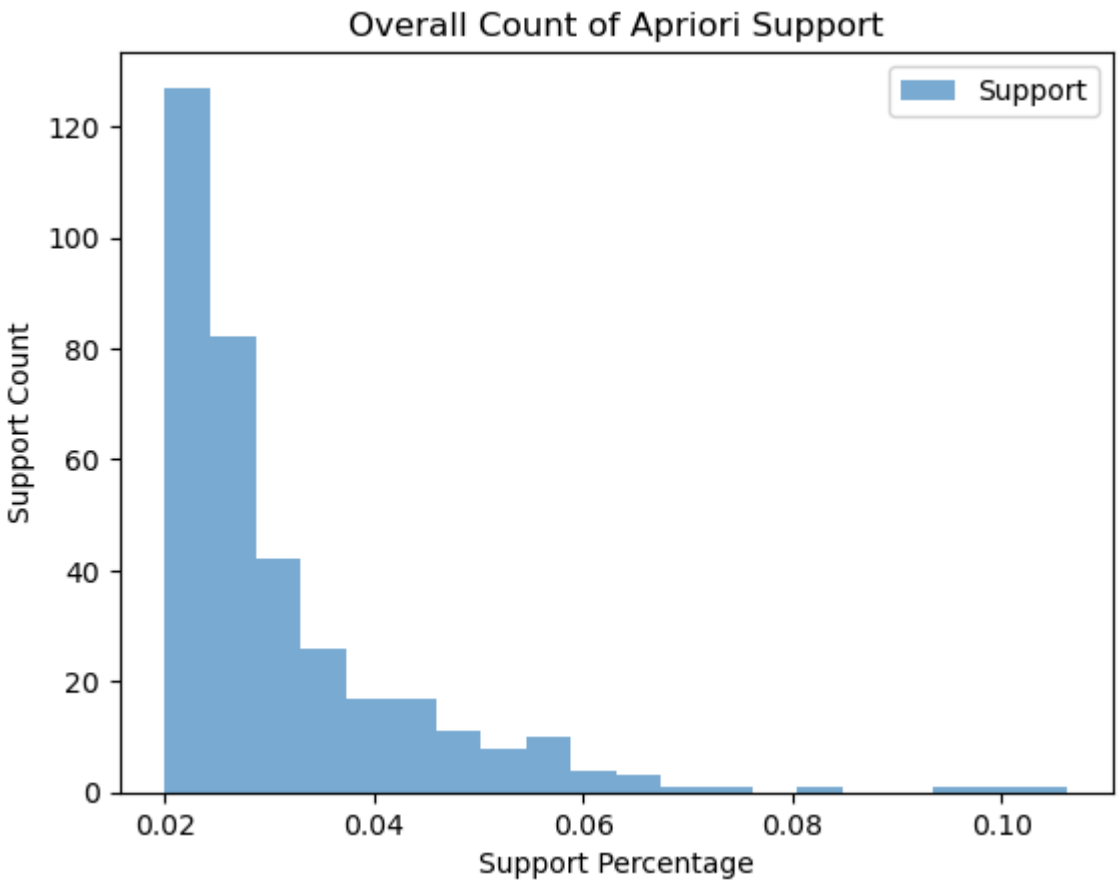
353 rows × 2 columns

The generated frequent itemsets show that only a limited number of product combinations occur frequently across transactions.

This reflects the sparse nature of customer purchasing behavior, where most products are purchased relatively infrequently.

To further show this, we've created a histogram over support count relative to the support percentage.

In [73]:
```
plt.hist(
    pattern['support'],
    bins=20,
    alpha=0.6,
    label="Support"
)


plt.xlabel("Support Percentage")
plt.ylabel("Support Count")
plt.title("Overall Count of Apriori Support")
plt.legend()
plt.show()
```



From the histogram and the look into the dataset above it, we can see that most frequent itemsets have support values close to the minimum threshold of 2%.

This indicates that broadly shared purchasing patterns are relatively rare, while only a small number of product combinations occur consistently across many transactions.

From the histogram we also see that the highest percentage is at 10%. We investigate the itemsets with percentage equal to or more than 6%.

In [74]:
```
pattern[(pattern['support'] >= 0.06)].sort_values(by=['support'], ascending=Fals
```

Out[74]:

	support	itemsets
271	0.106297	frozenset({85123A})
268	0.100941	frozenset({85099B})
112	0.095971	frozenset({22423})
230	0.081351	frozenset({47566})
11	0.075513	frozenset({20725})
258	0.070205	frozenset({84879})
95	0.067165	frozenset({22197})
159	0.066924	frozenset({22720})
32	0.063691	frozenset({21212})
106	0.062002	frozenset({22383})
13	0.061423	frozenset({20727})
116	0.060265	frozenset({22457})

The itemsets with support values above 6% consist primarily of individual products rather than larger product combinations.

This indicates that although globally frequent combinations of products are relatively rare, certain individual products appear consistently across a large number of transactions.

Many of the most frequent products appear to belong to decorative, kitchen, or gift-related categories, suggesting that these products are broadly popular across customers.

The relatively small difference between the most frequent products also indicates that customer purchasing behavior is highly distributed across many product types rather than dominated by a few extremely popular items.

To further investigate relationships between products, we now generate association rules in order to identify which products are frequently purchased together within the same transactions.

9.4 Association Rule Generation

The frequent itemsets identified in the previous section mainly consisted of individual products with relatively low support values. This indicates that globally dominant purchasing patterns are limited within the dataset.

To further investigate localized purchasing relationships between products, association rule mining is applied.

Association rules describe conditional relationships between products and help identify which products are likely to be purchased together within the same transaction.

The generated rules are evaluated using three standard metrics:

- **Support** measures how frequently a rule occurs.
- **Confidence** measures the probability that the consequent occurs when the antecedent occurs.
- **Lift** measures the strength of association compared to random co-occurrence.

Lift is especially important because confidence alone can be misleading when popular products appear frequently across many transactions. A lift value above 1 indicates a meaningful positive association.

Because the retail dataset contains many unique products and sparse transactions, support values are expected to remain relatively low. Therefore, the analysis focuses on identifying strong local relationships rather than globally frequent combinations.

A minimum confidence threshold of 0.5 was selected to prioritize reliable product relationships, while lift filtering removes associations that occur mainly due to product popularity.

Additional filtering is applied to improve rule quality and readability. A minimum support threshold removes extremely rare product combinations that may occur only by chance. Rule length is also checked to ensure that both antecedent and consequent contain at least one product, preventing incomplete or non-informative rules.

Finally, the rules are sorted by lift and confidence so that the strongest statistically meaningful associations are prioritized for interpretation.

In [80]:
```
# Generate association rules
rules = association_rules(pattern,metric='confidence',min_threshold=0.5)

# Additional quality filtering
rules = rules[
    (rules['lift'] > 1.2) &
    (rules['support'] > 0.01)
]

# Add rule length
rules['antecedent_len'] = rules['antecedents'].apply(len)
rules['consequent_len'] = rules['consequents'].apply(len)

# Keep meaningful rules
rules = rules[
    (rules['antecedent_len'] >= 1) &
    (rules['consequent_len'] >= 1)
]

# Sort by lift first, then confidence
rules = rules.sort_values(by=['lift', 'confidence'],ascending=False)

print("Number of final rules:", len(rules))
rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(20)
```

```
Number of final rules: 52
```

Out[80]:

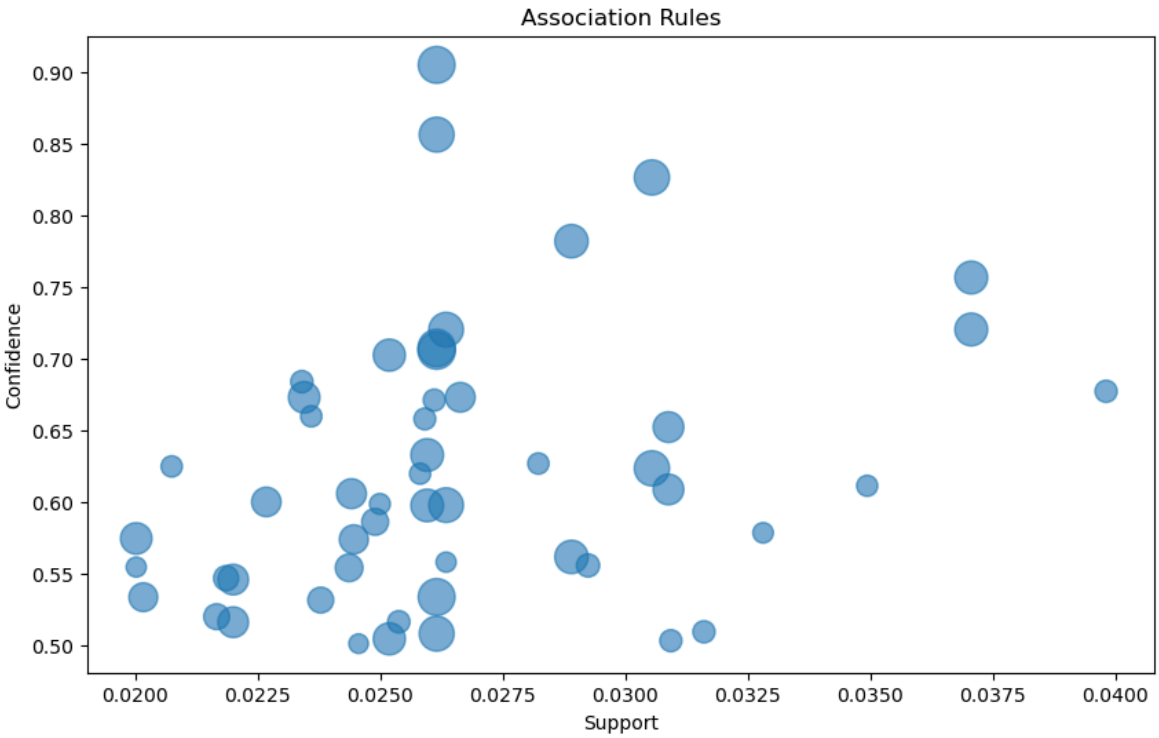
	antecedents	consequents	support	confidence	lift
23	frozenset({22698})	frozenset({22699, 22697})	0.026152	0.707572	19.094304
20	frozenset({22699, 22697})	frozenset({22698})	0.026152	0.705729	19.094304
24	frozenset({22697})	frozenset({22698, 22699})	0.026152	0.533990	18.475703
19	frozenset({22698, 22699})	frozenset({22697})	0.026152	0.904841	18.475703
9	frozenset({22698})	frozenset({22697})	0.030543	0.826371	16.873432
10	frozenset({22697})	frozenset({22698})	0.030543	0.623645	16.873432
22	frozenset({22699})	frozenset({22698, 22697})	0.026152	0.508443	16.646882
21	frozenset({22698, 22697})	frozenset({22699})	0.026152	0.856240	16.646882
18	frozenset({23300})	frozenset({23301})	0.026345	0.720317	16.351108
17	frozenset({23301})	frozenset({23300})	0.026345	0.598028	16.351108
13	frozenset({22699})	frozenset({22698})	0.028902	0.561914	15.203213
12	frozenset({22698})	frozenset({22699})	0.028902	0.781984	15.203213
2	frozenset({22697})	frozenset({22699})	0.037057	0.756650	14.710672
1	frozenset({22699})	frozenset({22697})	0.037057	0.720450	14.710672
27	frozenset({22630})	frozenset({22629})	0.025959	0.632941	14.575229
26	frozenset({22629})	frozenset({22630})	0.025959	0.597778	14.575229
32	frozenset({22356})	frozenset({20724})	0.025187	0.702557	14.081720
31	frozenset({20724})	frozenset({22356})	0.025187	0.504836	14.081720
41	frozenset({20723})	frozenset({20724})	0.023450	0.673130	13.491899
50	frozenset({20723})	frozenset({22355})	0.020024	0.574792	13.491018

After applying the filtering criteria, the number of rules is substantially reduced, leaving only the strongest purchasing relationships.

This confirms that although the dataset contains highly diverse purchasing behavior, meaningful local product associations still emerge. These rules typically represent complementary products, product variants, or items belonging to the same thematic collection.

Sorting by lift rather than confidence highlights relationships that are statistically stronger than expected by chance, making the final rules more informative for recommendation systems and basket analysis.

In [76]:
```
# Support vs confidence scatter plot
plt.figure(figsize=(10,6))

plt.scatter(
    rules['support'],
    rules['confidence'],
    s=rules['lift'] * 20,
    alpha=0.6
)

plt.xlabel('Support')
plt.ylabel('Confidence')
plt.title('Association Rules')

plt.show()
```



The scatter plot demonstrates that most association rules have relatively low support values but still maintain moderate to high confidence. This indicates that strong purchasing relationships exist even when the product combinations themselves are not globally frequent.

The varying point sizes additionally show that some rules possess substantially stronger associations than others when compared to random chance.

9.5 Readability

To better understand our found association rules, we map our StockCodes back to their descriptions.

This allows us to better read and understand the outcome of our rules.

In [ ]:
```
# Map StockCodes back to descriptions for readability
def fmt(itemset):
```

```
    return ", ".join(product_map.get(i, str(i)) for i in itemset)

print("Top association rules (by lift):\n")
for _, row in rules.head(10).iterrows():
    print(f"IF:   {fmt(row['antecedents'])}")
    print(f"THEN: {fmt(row['consequents'])}")
    print(f"      support={row['support']:.3f}, confidence={row['confidence']:.3
    print()
```

```
Top association rules (by lift):

IF:   PINK REGENCY TEACUP AND SAUCER
THEN: ROSES REGENCY TEACUP AND SAUCER , GREEN REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.708, lift=19.09

IF:   ROSES REGENCY TEACUP AND SAUCER , GREEN REGENCY TEACUP AND SAUCER
THEN: PINK REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.706, lift=19.09

IF:   GREEN REGENCY TEACUP AND SAUCER
THEN: PINK REGENCY TEACUP AND SAUCER, ROSES REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.534, lift=18.48

IF:   PINK REGENCY TEACUP AND SAUCER, ROSES REGENCY TEACUP AND SAUCER
THEN: GREEN REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.905, lift=18.48

IF:   PINK REGENCY TEACUP AND SAUCER
THEN: GREEN REGENCY TEACUP AND SAUCER
      support=0.031, confidence=0.826, lift=16.87

IF:   GREEN REGENCY TEACUP AND SAUCER
THEN: PINK REGENCY TEACUP AND SAUCER
      support=0.031, confidence=0.624, lift=16.87

IF:   ROSES REGENCY TEACUP AND SAUCER
THEN: PINK REGENCY TEACUP AND SAUCER, GREEN REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.508, lift=16.65

IF:   PINK REGENCY TEACUP AND SAUCER, GREEN REGENCY TEACUP AND SAUCER
THEN: ROSES REGENCY TEACUP AND SAUCER
      support=0.026, confidence=0.856, lift=16.65

IF:   GARDENERS KNEELING PAD CUP OF TEA
THEN: GARDENERS KNEELING PAD KEEP CALM
      support=0.026, confidence=0.720, lift=16.35

IF:   GARDENERS KNEELING PAD KEEP CALM
THEN: GARDENERS KNEELING PAD CUP OF TEA
      support=0.026, confidence=0.598, lift=16.35
```

9.6 Interpretation of Association Rules

The strongest association rules primarily involve products belonging to similar product collections, decorative themes, or complementary categories. This indicates that customers often purchase related items within the same transaction.

For example, customers purchasing *ROSES REGENCY TEACUP AND SAUCER* are also likely to purchase *GREEN REGENCY TEACUP AND SAUCER*. This suggests that customers frequently purchase coordinated product variants from the same collection.

Many of the generated rules reveal relationships between:

- similar product variants,
- thematically related products,
- and complementary decorative items.

This indicates that customer purchasing behavior is influenced by:

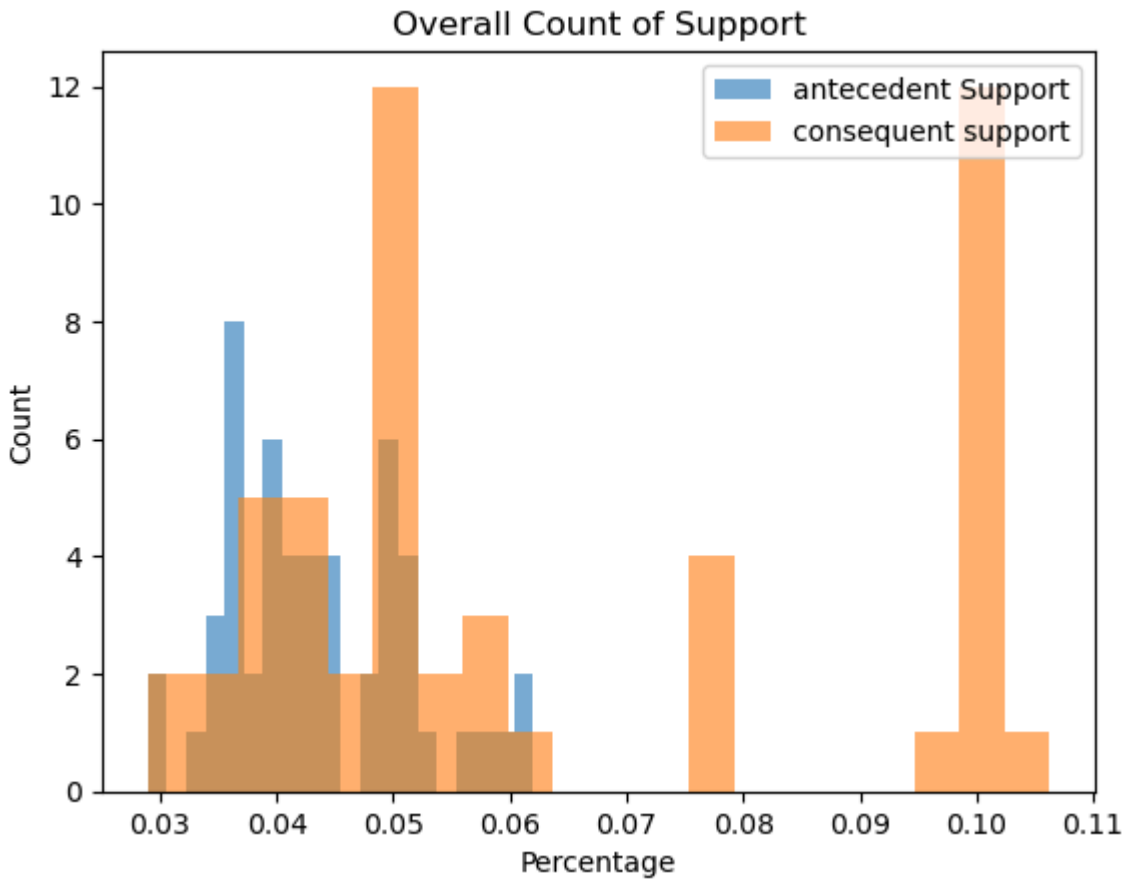
- thematic consistency,
- product similarity,
- and complementary preferences.

The support values remain relatively low across most rules, which reflects the high diversity of products within the retail dataset. Although individual product combinations are not globally frequent, several rules still achieve high confidence values.

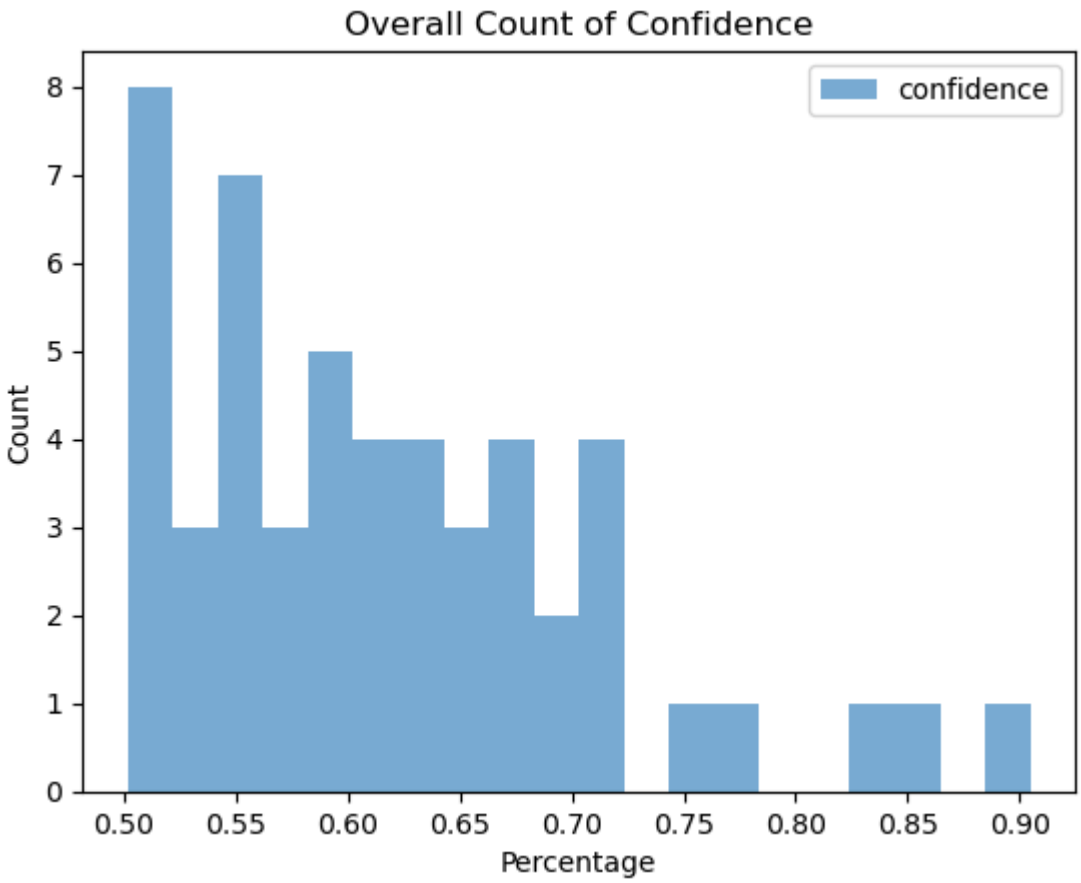
This indicates that meaningful localized purchasing patterns exist even in a sparse transactional environment.

The following histograms support the interpretation of the generated association rules by illustrating the overall distribution of support and confidence values.

In [ ]:
```
plt.hist(
    rules['antecedent support'],
    bins=20,
    alpha=0.6,
    label="antecedent Support"
)
plt.hist(
    rules['consequent support'],
    bins=20,
    alpha=0.6,
    label="consequent support"
)


plt.xlabel("Percentage")
plt.ylabel("Count")
plt.title("Overall Count of Support")
plt.legend()
plt.show()
```



In [79]:
```
plt.hist(
    rules['confidence'],
    bins=20,
    alpha=0.6,
    label="confidence"
)

plt.xlabel("Percentage")
plt.ylabel("Count")
plt.title("Overall Count of Confidence")
plt.legend()
plt.show()
```



The support distributions show that both antecedent and consequent itemsets generally occur at relatively low frequencies within the dataset. Most support values remain below 10%, which reflects the high diversity of products and indicates that most purchasing relationships are localized rather than globally dominant patterns.

The confidence distribution shows that many rules still achieve moderate to high confidence despite low support. This indicates that while product combinations may occur infrequently overall, certain products act as strong indicators for additional related purchases.

Overall, the association rule analysis complements the previous frequent itemset analysis by uncovering localized product relationships that are not immediately visible through support values alone.

While globally frequent purchasing patterns remain limited, the generated rules demonstrate that strong conditional relationships still exist between many related products.

The slightly higher consequent support suggests that some products appear repeatedly as common follow-up purchases across multiple association rules.

9.7 Comparison with Graph Mining

The graph-based analysis and the Apriori-based pattern mining analysis provide complementary perspectives on customer purchasing behavior.

The graph analysis focused on identifying large-scale community structures among products through co-occurrence networks.

In contrast, the Apriori analysis identifies explicit transactional relationships between products using frequent itemsets and association rules.

Both analyses consistently indicate that:

- customer purchasing behavior is highly diverse,
- globally dominant purchasing patterns are limited,
- and strong local relationships exist between related products.

Together, the two approaches provide a broader understanding of product relationships within the dataset.

10. Final Synthesis and Reflection

- **Key insights**:
    - Most transactions with highest overall price and quantities are from the UK (which makes sense given this is a UK store).
    - Some transactions of other countries have high price and quantity (belonging to the yellow cluster), which is a good place to start if company wants to be international.
    - Outliers seemed to be alot in this dataset. However, When investigating it per community we see some having more outliers than other, but overall invoices are normal.
    - There are 5 large communities whithin the purchased products, when investigating one of subclusters we see overlap. This should help companies when sorting their products according to these communities, and maybe for when recommending related products for customers.
    - The community size is related to whether an item is regularly bought (large communities), or part of a more nieche collection (small communities).
    - Through Association rules mining we found with high confidence that customers who purchase a product, are more likely to purchase another product with similar attribute, has thematic consistency or are complementary decoration for the original product.
    - Customer purchasing behavior is diverse because of the diverse category of products, and so any patterns concluded here are not necessarily applied for all purchasing pattern. This is supported by the low value of support.
- **Limitations**:
    - Not much data could be used for higher dimensional clustering. Chosen variables, when visualized, didn't seem to have concrete clustering result.
    - Very hard to implement Clique-proculation method on the entire dataset (and even on the biggest louvian cluster), since it takes $O(exp(n))$.

    - The diversity of the products affected the results when doing pattern mining. Minimum support needed to be lowered a lot in order to get meaningful results.
- **Revisions after feedback**:

10.1 Feedback from checkpoint 1

Preprocessing

- For the Categorical Features Overview, we added two graphs for top 10 customers by transaction count and products by transaction count to help better understand the data distribution.
- Added section going over missing data, and how we handle it/why we chose to ignore certain missing data points.
- Added ending section in Sparcity/Density section going over the results, discussing what we expected to see and how it relates to the scope of our project.

Analysis Quality (Module 1)

- For Clustering algorithm, we removed the parts where we used `Country` as a clustering feature since it did not give interpretable results. `NumUniqueProducts` was excluded because it captures basket size in a way that overlaps substantially with `TotalQuantity`, introducing redundancy into the feature space. Furthermore, when additional derived features such as `AvgPrice` and `AvgQuantityPerProduct` were included alongside it, the silhouette scores became uniformly high across all values of k. This is a sign that clustering was being driven by feature redundancy rather than genuine structure in the data, making the results misleading and difficult to interpret.
- For the Anomaly detection part we firstly added better clarifications and explanations. Then for the analysis we added a sections going further into depth of the analysis in regards to our initial objective.

10.2 Feedback from checkpoint 2:

Graph Mining

- Added clique-proculation as an extra algorithm for Graph Mining to compare with the Louvain method during the community detection (section 8.8). We had some trouble being able to run this method on the full graph, and ended up having to reduce it down by a lot (300 nodes) for us to be able to run it.
- Added a threshold sensitivity analysis section to support the threshold choice made during the filtering (section 8.2.1).
- Added a section about degree distribution of the filtered graph to further characterize its structure (section 8.2.2).
- Added extra graph statistics such as density before/after filtering and global clustering coefficient.

- Added a comparison between the communities and the anomalies section to look for more interesting findings (section 8.9).
- Improved upon explanations and general language thoughtout the entire Graph Mining section.